TECHNICAL UNIVERSITY – SOFIA
Fakultät für deutsche Ingenieur- und
Betriebswirtschaftsausbildung

Leksell Frame: Actuation and Control

Bachelor's Thesis

PREPARED BY

Aleksandra Naydenova

Matriculation Number: 901222015

Field of Study: Mechatronics and Information Technology

UNDER THE SUPERVISION OF

Assoc. Prof. Stanislav Enev

5. Mai 2026

TECHNICAL UNIVERSITY – SOFIA
Fakultät für deutsche Ingenieur- und
Betriebswirtschaftsausbildung

Date of assignment: 16.02.2026
Date of delivery: 17.04.2026

Approved: ……
(Prof. D. Sc. Marin Marinov)
Dean of FDIBA

ASSIGNMENT

for the development of a **Bachelor's Thesis**

Thesis: Leksell Frame Actuation and Control
Thema: Antrieb und Steuerung des Leksell-Rahmens
Тема: Задвижване и управление на рамка на Лексел

Student: **Aleksandra Dimitrova Naydenova**
Subject area: **Mechatronics and Information Technology** Faculty Nr.: **901222015**

The objective of this work is to design and prototype the actuation and control system for the Y- and Z-axes of a Leksell stereotactic frame, based on a system-level approach to motion control architecture. The actuation of both axes will be implemented using stepper motors.
The primary focus of the work is the development of firmware for the motion control units governing each axis. A CAN-enabled solution is proposed, supporting multiple motion control modes and providing a CAN interface for operation control, monitoring, and configuration.
The system will be implemented using a Microchip PIC18F26K83 microcontroller, stepper motors, and custom-designed control board electronics.

Structure:

1. Introduction and problem formulation
2. Motor control at the application level: stepper motor drivers and control strategies
3. CAN interface design and implementation: PIC18F26K83 CAN controller programming model and API
4. Motion controller firmware design and development
5. Design and prototyping of the mechanical actuation support structure; experimental validation
6. Conclusion

Supervisor of Bachelor's thesis:
…….
(Prof. Dr. Stanislav Enev)

Student: ……

Declaration

I hereby declare that I have written this Bachelor's thesis independently and without external help, that I have not used any sources or aids other than those specified, and that I have marked the sections taken literally or in analogy as such. The drawings or illustrations in this thesis have been created by myself or have been provided with an appropriate reference. I certify that I have not previously submitted an examination paper on the same or a similar subject to any examination authority or other higher education institution.

Sofia, 21. Juni 2026 Aleksandra Naydenova

Abstract

This bachelor thesis presents the development of a distributed embedded control system for the automation of selected axes of a stereotactic frame used in neurosurgical positioning applications. The work focuses on the implementation of precise motor-control functionality, communication between distributed controller nodes and reliable positioning through encoder-based feedback and homing procedures.

The developed system is based on a modular master–slave architecture using PIC18F26K83 microcontrollers and CAN bus communication. A master controller transmits motion-related commands to dedicated axis nodes responsible for local axis control. Each slave node manages motor actuation, encoder feedback processing and execution of motion-control algorithms independently. The communication layer was implemented using the integrated ECAN peripheral of the microcontroller together with a custom application-level protocol designed for compact command and status exchange.

A modular firmware architecture was implemented using separate software modules for axis control, motor control, encoder processing, communication handling and configuration management. The axis controller was developed as a finite state machine supporting movement execution, homing, controlled stopping, emergency stopping and fault handling. Position feedback is provided through an encoder-based measurement system, while operational parameters are stored in internal EEPROM memory and can be modified through the communication interface.

The communication layer was implemented using the integrated ECAN peripheral of the PIC18F26K83 together with a custom application-level protocol supporting command transmission, status reporting and configuration management. The distributed architecture enables independent axis operation while maintaining centralized supervision through the master controller.

Experimental validation was performed using dedicated test setups to verify motion-control behavior, homing procedures, encoder-based positioning, CAN communication and EEPROM parameter storage. The obtained results demonstrate reliable operation of the developed firmware architecture and confirm the suitability of the proposed solution for distributed embedded motion-control applications requiring modularity, scalability and repeatable positioning performance.

Table of Contents

List of Abbreviations

ACK	Acknowledgement
API	Application Programming Interface
CAN	Controller Area Network
CAN_H	CAN High line
CAN_L	CAN Low line
CMD	Command
CMM	Coordinate Measuring Machine
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DLC	Data Length Code
DIR	Direction
ECAN	Enhanced Controller Area Network
EEPROM	Electrically Erasable Programmable Read-Only Memory
EN	Enable
EOF	End of Frame
FSM	Finite State Machine
GPIO	General Purpose Input/Output
IDE	Identifier Extension
ISR	Interrupt Service Routine
MCU	Microcontroller Unit
PCB	Printed Circuit Board
PSU	Power Supply Unit
RTR	Remote Transmission Request
SEQ	Sequence
SOF	Start of Frame
SPI	Serial Peripheral Interface
STEP	Step Signal
UART	Universal Asynchronous Receiver-Transmitter
PWM	Pulse Width Modulation

List of Figures

List of Tables

List of Listings

1. Introduction. Problem Formulation.

1.1. Background

Stereotactic frames are precision mechanical systems used mainly in neurosurgical procedures to define spatial coordinates relative to a patient or a target region. Accurate positioning is essential because the frame serves as a reference structure during the intervention. One of the most widely used systems of this type is the Leksell stereotactic frame, where positioning accuracy, repeatability and reliability are of high importance. The frame operates according to the center-of-arc principle, which allows access to intracranial targets from different trajectories while maintaining precise positioning. [1]

Figure 1 Leksell stereotactic frame used for neurosurgical positioning [2]

In conventional operation, positioning of the frame elements is performed manually. Although this approach is functional, repeated adjustments can be time-consuming and may introduce small positioning inconsistencies. These limitations motivate the development of an automated actuation system capable of improving repeatability and simplifying operation.

1.2. Problem Statement

The main objective of this work is the development of an actuation and control system for the Y- and Z-axes of a Leksell stereotactic frame. The system must support controlled and repeatable positioning, homing and reference detection, while maintaining safe operation under different operating conditions.

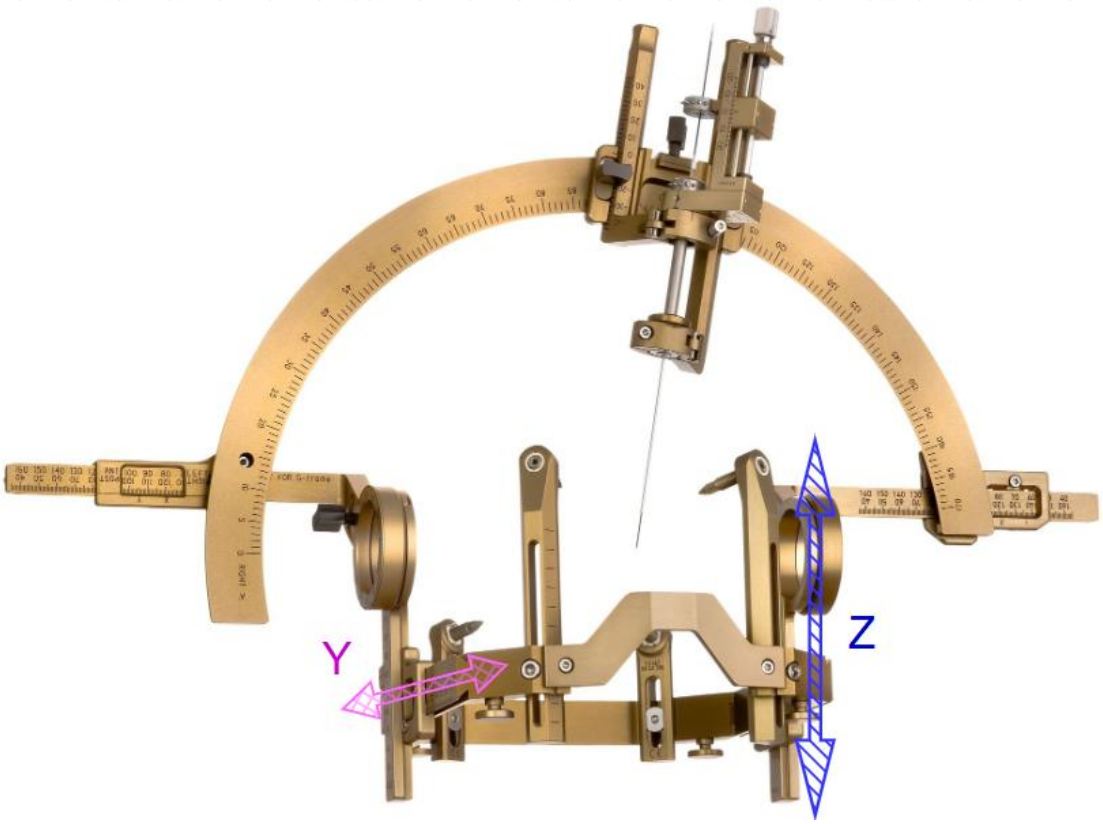

Figure 2 Defined translational axes of the stereotactic frame subject to motorization in the following work

One of the main challenges is integrating the actuation system without affecting the original mechanical structure and calibration accuracy of the frame. Since stereotactic systems rely heavily on precise alignment, permanent structural modifications could negatively affect positioning accuracy. For this reason, the developed actuation mechanism is designed as a non-invasive external attachment that can be mounted without permanent modification of the frame.

In addition to the mechanical constraints, the system must also address several motion-control challenges, including position tracking, reference detection and reliable stopping behavior. These requirements necessitate a structured embedded firmware solution integrating motor control, feedback processing and safety supervision.

1.3. Aim and Objectives

The aim of this work is to design and prototype an actuation and control system for the Y- and Z-axes of a Leksell stereotactic frame, with a primary focus on the development of embedded motion-control firmware.

The main objectives of the work are:

- to analyse the requirements for axis positioning in stereotactic systems;
- to design a modular firmware architecture for motion control;

- to implement movement, homing and stopping algorithms;
- to integrate encoder-based feedback for position estimation and motion verification;
- to implement distributed communication between master and slave nodes over CAN;
- to prepare the system for hardware implementation and experimental validation.

1.4. Scope of the Work

The developed system consists of several functional modules, including motor control, encoder-based feedback and motion detection. These modules are integrated within an axis-control layer implemented as a finite state machine responsible for movement execution, homing and fault handling.

The firmware follows a distributed control structure. High-level commands are processed by a master node and transmitted over a CAN bus to slave nodes responsible for local axis control.

The scope of this work does not include clinical validation or final medical certification of the system. The primary focus is the development of a functional prototype and a modular firmware architecture suitable for further refinement and testing.

2. Motor Control Basics at Application Level

2.1. Stepper Motor Principles

Stepper motors are electromechanical actuators that convert sequences of electrical excitation into discrete mechanical motion. Unlike conventional DC motors, which rotate continuously when voltage is applied, stepper motors operate by moving in predefined angular increments referred to as steps. This property enables precise position and speed control using digital control signals generated by a microcontroller.

The operating principle of a stepper motor is based on the interaction between the stator magnetic field and the rotor. The stator consists of multiple windings arranged in phases, while the rotor is typically constructed from permanent magnets or toothed ferromagnetic material. By sequentially energizing the stator windings, a rotating magnetic field is produced, causing the rotor to align with the active magnetic poles. Repeating this process generates continuous rotational motion in discrete angular increments. The rotor moves toward the equilibrium position created by the energized stator poles. Each commutation state creates a new magnetic field orientation, forcing the rotor to advance to the next stable angular position. [3]

A bipolar stepper motor requires controlled current flow through its windings in both directions. Reversing the current polarity changes the orientation of the magnetic field generated by the coil, allowing bidirectional rotation and controlled commutation sequences.

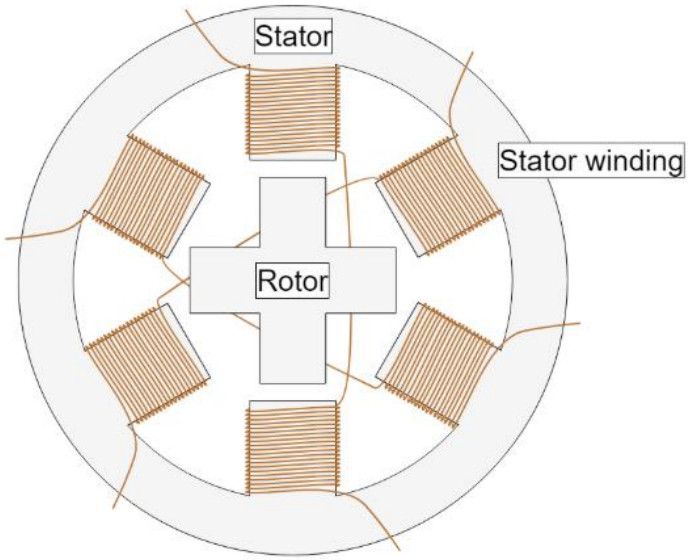

Figure 3 Simplified structure of a bipolar stepper motor showing the rotor, stator and phase windings. [4]

The angular displacement of the motor shaft is directly proportional to the number of excitation steps applied to the windings. Similarly the rotational speed depends on the frequency of the excitation sequence generated by the controller.

The relation between stepping frequency and angular velocity can be expressed as:

$$\omega = \frac{\theta_{step} \cdot f_{step}}{2\pi}$$

where:

- ω is the angular velocity,
- θ_{step} is the motor step angle,
- f_{step} is the excitation frequency.

This relationship allows straightforward digital control of motor speed by adjusting the timing of the excitation sequence.

2.2. Stepper Motor Drivers and Control

Stepper motor windings cannot be driven directly from a microcontroller because the current required for coil energization exceeds the output capability of standard GPIO pins. For this reason an intermediate power-driving stage is required between the controller and the motor windings.

In conventional embedded motion-control systems this functionality is often implemented using dedicated stepper motor driver modules that provide high-level interfaces such as STEP and DIR signals. These drivers internally manage phase commutation, current regulation and winding excitation.

In the developed system a different approach is adopted. Instead of using a dedicated high-level stepper driver that accepts STEP and DIR signals, motor phase commutation is controlled directly by the firmware. The required winding currents are generated through a dual H-bridge driver integrated circuit (TB6612FNG), while the microcontroller determines the excitation sequence and updates the bridge control signals accordingly. This approach allows the firmware to manage phase commutation and excitation sequencing directly.

2.2.1. H-Bridge Principle

To produce rotational motion in a bipolar stepper motor current must flow through the windings in both directions. Reversing the current polarity reverses the magnetic field generated by the coil, allowing bidirectional rotor movement.

This functionality is achieved using an H-bridge circuit. An H-bridge consists of four switching elements arranged in a configuration that enables current flow through the winding in either direction.

A bipolar stepper motor contains two separate phase windings. Since current reversal is required for each individual winding, two H-bridge circuits are required for complete motor operation. Each H-bridge controls the direction of current flow through one motor phase. [5]

By selectively activating different pairs of switches the controller determines the polarity applied to the motor winding:

- one switch combination produces positive current flow;
- the opposite combination produces reversed current flow.

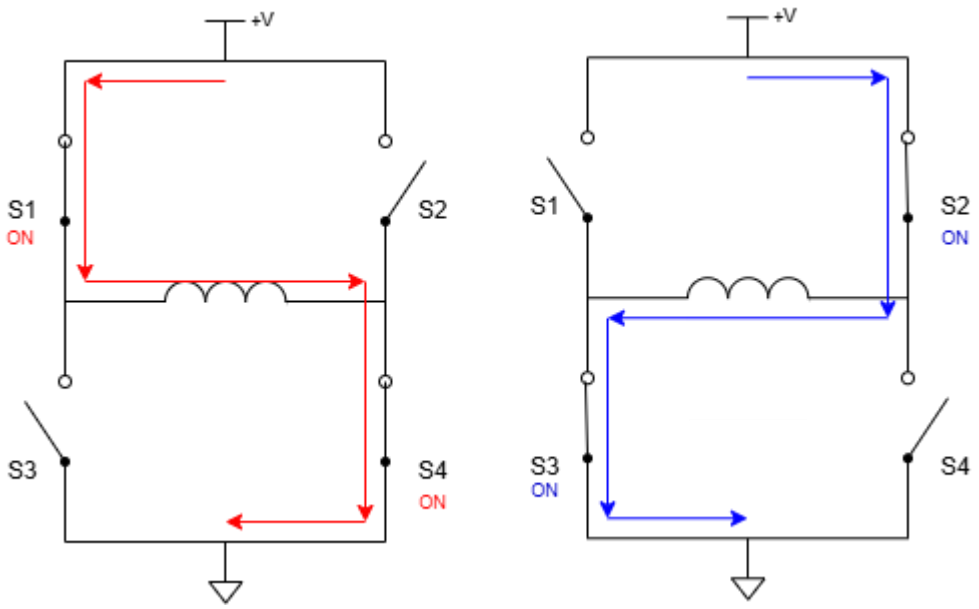

Figure 4 Simplified H-bridge current reversal for a bipolar winding

Figure above illustrates the principle of current reversal through a single winding of a bipolar stepper motor using an H-bridge configuration. By activating opposite diagonal switch pairs, current can be forced through the winding in either direction, producing opposite magnetic field orientations. Only one diagonal pair may be active at a time, since conflicting switch combinations create a direct short circuit between the supply rails. In the developed system, the TB6612FNG dual H-bridge driver serves as the interface between the microcontroller outputs and the motor windings. The driver provides the power-switching circuitry required to reverse the current through each motor phase, while the excitation sequence is generated by the firmware.

2.2.2. Software-Controlled Commutation

The developed motion-control system implements software-controlled commutation, where the microcontroller directly generates the excitation sequence required for motor operation.

Instead of generating STEP and DIR signals for an external driver, the firmware explicitly controls the bridge outputs associated with each motor phase. The active winding state are updated sequentially to generate rotational motion.

A simplified bipolar excitation sequence is shown below:

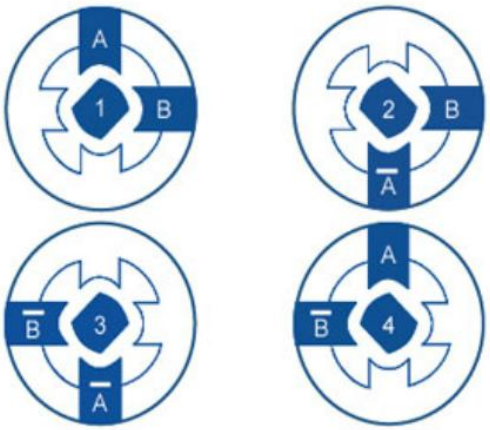

Figure 5 Full-step excitation sequence for bipolar stepper motor [6]

The order of the excitation sequence determines the direction of rotation, while the timing between successive commutation states determines the rotational speed.

The commutation sequence is generated periodically using timer-controlled firmware routines. Hardware timers provide periodic timing for motor commutation independent of the main control loop execution.

This approach provides complete software-level control over:

- phase excitation order;
- rotational direction;
- stepping frequency;
- motor enable and disable behavior.

2.2.3. Full-Step and Half-Step Excitation Modes

Stepper motors may be operated using different excitation strategies depending on the required positioning resolution, smoothness and torque characteristics.

In full-step excitation mode the motor advances by one complete step angle during each commutation cycle. This mode provides relatively high holding torque and simple implementation, making it suitable for many embedded positioning applications.

In half-step mode additional intermediate excitation states are inserted between full steps by alternating single-phase and dual-phase activation. This effectively doubles the positioning resolution and reduces motion discontinuities.

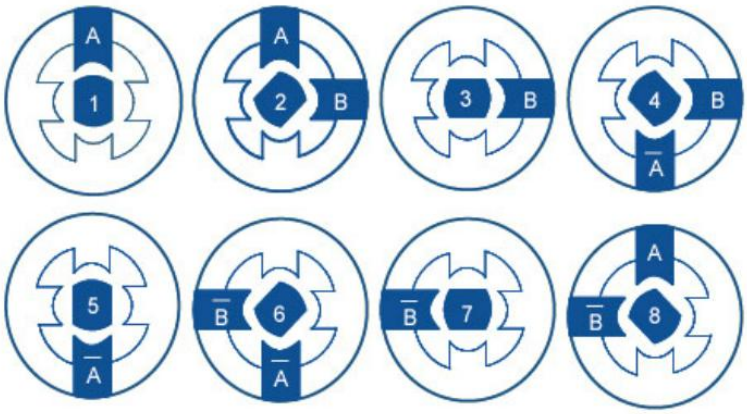

Figure 6 Half-step excitation sequence [6]

Although half-step operation improves smoothness and angular resolution, it also increases commutation complexity and timing requirements.

Direct software-controlled commutation allows complete control over phase excitation, stepping frequency and motor direction. This approach integrates naturally with the developed firmware architecture and simplifies implementation of custom excitation sequences. The main limitations are the increased firmware complexity and the lack of advanced features such as current regulation and microstepping that are commonly available in dedicated motor-driver solutions.

2.3. Motion Control Concepts

2.3.1. Position and Speed Control

In embedded motion-control systems two of the most important controlled quantities are position and rotational speed. In stepper motor systems, these quantities are directly related to the excitation sequence generated by the controller firmware. [7]

Position control is achieved by controlling the number of commutation steps applied to the motor windings. Each excitation step produces a fixed angular displacement of the rotor.

By accumulating these incremental movements, the controller determines the relative position of the motor shaft.

The angular position of the motor can therefore be expressed as:

$$\theta = N \cdot \theta_{step}$$

where:

- θ is the total angular displacement,
- N is the number of executed steps,
- θ_{step} is the step angle of the motor.

This direct relationship between excitation steps and mechanical displacement simplifies position control and makes stepper motors particularly suitable for embedded positioning systems.

Speed control is achieved by adjusting the frequency of the commutation sequence. Increasing the excitation frequency produces faster rotation, while reducing the excitation rate slows the motor down. In embedded systems this functionality is commonly implemented using hardware timers that periodically trigger phase commutation.

Although stepper motors can theoretically start and stop instantaneously, practical systems must consider the mechanical inertia of the rotor and load. Sudden changes in stepping frequency may cause the motor to lose synchronization, resulting in missed steps or unstable motion. For this reason acceleration and deceleration profiles are commonly implemented.

2.3.2. Open-Loop Control

Stepper motor systems are frequently operated in open-loop control mode. In this approach the controller assumes that each commanded excitation step is executed successfully by the motor. Since no direct position feedback is required, the control structure remains relatively simple and computationally efficient.

However open-loop control also introduces several limitations. Under conditions such as excessive load torque, rapid acceleration or mechanical obstruction, the motor may fail to follow the commanded excitation sequence. This phenomenon, commonly referred to as missed steps, leads to position errors that cannot be directly detected without additional sensing mechanisms. [7]

Despite these limitations open-loop stepper control remains widely used in embedded applications due to its simplicity, low implementation cost and straightforward integration with digital control systems.

2.3.3. Hybrid Feedback-Assisted Control

To improve operational reliability, the developed system supplements open-loop commutation control with encoder-based feedback mechanisms.

The encoder is used primarily for:

- position estimation;
- motion verification;
- stop detection during homing - stalling;
- fault monitoring.

Unlike fully closed-loop servo systems, the implemented approach does not continuously regulate motor position using feedback correction. Instead the feedback mechanisms are integrated to enhance reliability of motion supervision and reference detection.

This hybrid approach combines the simplicity of stepper motor control with additional verification capabilities, improving robustness without significantly increasing system complexity.

2.3.4. Application in the Developed System

In the developed firmware architecture motion control is implemented through cooperation between the motor control module, encoder module and axis control logic.

The control system performs:

- generation of commutation sequences;
- control of excitation timing;
- direction management;
- monitoring of encoder feedback;
- execution of homing procedures;
- detection of abnormal motion conditions.

These functions are coordinated by the axis control module, which operates as a finite state machine and periodically updates the system behavior based on commanded targets and feedback information.

3. CAN Bus Basics and Implementation

3.1. Overview of CAN

The Controller Area Network (CAN) is a serial communication protocol widely used in distributed embedded systems. It was originally developed for automotive applications, but is now commonly used in industrial control, robotics and embedded motion systems due to its reliability and resistance to electrical noise. [8]

In the developed system CAN is used for communication between the master controller and the distributed slave nodes responsible for axis control. The protocol was selected because it allows multiple devices to share the same communication bus while keeping wiring relatively simple.

CAN communication uses two differential signal lines: CAN_H and CAN_L. Differential signaling improves immunity to electromagnetic interference, which is important in systems operating near motors and switching electronics. Termination resistors are placed at both ends of the bus to reduce signal reflections.

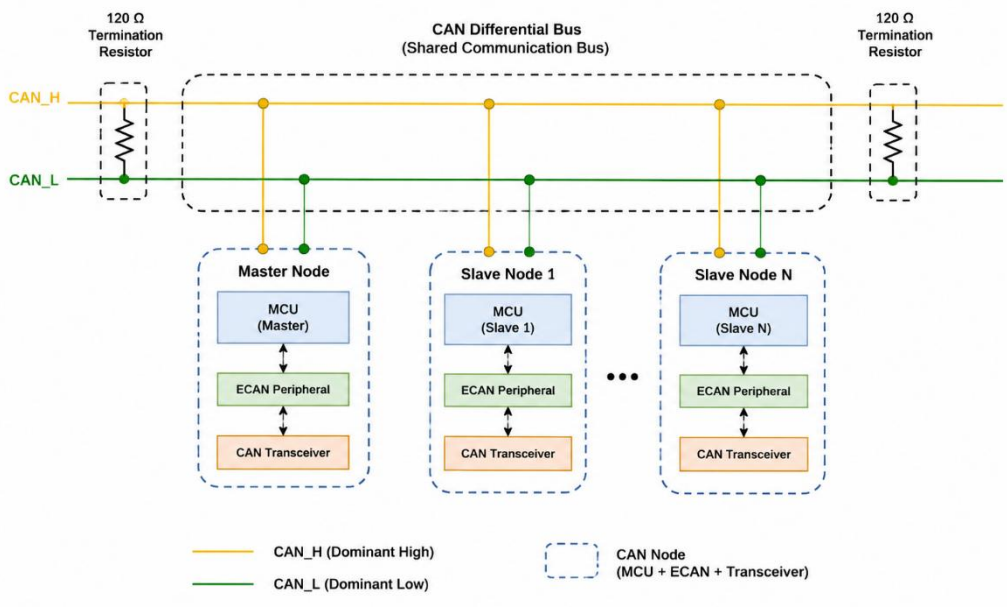

Figure 7 Simplified CAN bus topology used in the developed distributed control system

Each CAN node consists of a microcontroller, a CAN controller and a CAN transceiver. The CAN controller manages frame transmission, arbitration and error detection, while the transceiver converts logic-level signals into differential bus signals.

CAN is a message-oriented protocol. Instead of using device addresses, communication is performed through message identifiers. All nodes receive the transmitted frame and determine locally whether the message should be processed.

One of the main advantages of CAN is its arbitration mechanism. If multiple nodes start transmission simultaneously, the message with the highest priority continues without collisions. In the developed system this behavior is important because emergency-stop messages must take priority over standard motion commands. CAN also includes built-in error detection mechanisms such as CRC (cyclic redundancy check) verification, acknowledgment monitoring and automatic retransmission. These features improve communication reliability in distributed embedded systems.

3.2. CAN Communication Principles and Frame Structure

Communication in a CAN network is performed through structured data frames transmitted over a shared communication bus. All nodes connected to the network monitor the bus continuously and may begin transmission when the bus becomes idle. [9]

A CAN frame contains an identifier, control information and a data field. The identifier defines both the meaning of the message and its priority during arbitration. Lower identifier values correspond to higher priority.

The standard CAN data frame used in classical CAN is shown below:

```
| SOF | Identifier (11-bit) | RTR | IDE | DLC | DATA (0–8 bytes) | CRC | ACK | EOF |
```

Listing 1 Standard CAN data frame

The frame consists of several fields:

- Start of Frame (SOF) - A dominant bit indicating the beginning of a new frame and synchronizing all nodes on the bus.
- Identifier - An 11-bit value that defines both the message content and its priority during arbitration.
- Remote Transmission Request (RTR) - Indicates whether the frame contains data (data frame) or requests data (remote frame).
- Identifier Extension (IDE) - Distinguishes between standard (11-bit) and extended (29-bit) identifiers.
- Data Length Code (DLC) - Specifies the number of data bytes contained in the frame (0 to 8 bytes).
- Data Field - Contains the actual payload, with a maximum size of 8 bytes in classical CAN.

- Cyclic Redundancy Check (CRC) - Used for error detection by verifying data integrity.
- Acknowledgment (ACK) - Indicates successful reception of the frame by at least one node.
- End of Frame (EOF) - Marks the end of the message.

In classical CAN the data field can contain up to 8 bytes. This limitation influenced the design of the communication protocol used in the developed system. All commands and responses were intentionally designed to fit within a single CAN frame in order to simplify synchronization and avoid message fragmentation.

CAN communication is based on dominant and recessive logic levels. During arbitration nodes transmit and monitor the bus simultaneously. If a node transmits a recessive bit but detects a dominant bit, it immediately stops transmission and waits for the next available bus cycle.

This arbitration mechanism allows higher-priority messages to be transmitted without corruption or collisions. In the developed system emergency-related commands are assigned higher-priority identifiers than standard motion-control messages.

CAN also includes several built-in error detection mechanisms. CRC verification is used to detect corrupted data, while the acknowledgment field confirms successful frame reception by at least one node on the network. If an error is detected, the frame is retransmitted automatically.

3.3. Implementation on the PIC18F26K83 ECAN Peripheral

The developed communication system is implemented using the integrated Enhanced Controller Area Network (ECAN) peripheral of the PIC18F26K83 microcontroller. The ECAN module provides hardware support for CAN frame transmission, reception, arbitration and error handling, reducing the software overhead required for communication management. [10]

The microcontroller cannot be connected directly to the CAN bus lines. For this reason, an external CAN transceiver is used between the ECAN peripheral and the physical CAN bus. The transceiver converts the logic-level TX and RX signals of the microcontroller into the differential CAN_H and CAN_L bus signals.

The ECAN peripheral supports:

- transmission and reception buffers;
- hardware arbitration;
- automatic CRC generation;
- error detection and retransmission;

- message filtering and masking.

Hardware receive filters are used so that each slave node processes only messages intended for its assigned CAN identifier. The ECAN peripheral is initialized during system startup by configuring the communication baud rate, transmit and receive buffers, and acceptance filters. Low-level transmission and reception functions are provided by the MCC-generated driver layer and are used by the higher-level communication protocol described in the following section.

3.4. Application-Level Communication Protocol

Although the CAN protocol defines the structure of the transmitted frame, the meaning of the transmitted data must be defined by the application firmware. For this reason a custom application-level communication protocol was implemented for communication between the master controller and the axis nodes.

The protocol follows a command-response structure. The master controller transmits commands related to axis control, while the slave node executes the requested operation and returns a response frame containing status information.

Table 1 Command Frame (Master → Slave)

BYTE	**CONTENT**
0	Command identifier
1	Sequence number
2-5	Parameter (float, e.g. position in mm)
6-7	Reserved

The command identifier specifies the requested operation, such as:

- homing;
- movement command;
- controlled stop;
- emergency stop;
- fault reset.

The parameter field is used mainly for movement-related commands and contains values such as target position in millimeters.

The slave node responds using a status frame containing execution results and current axis information.

Table 2 Response Frame (Slave → Master)

BYTE	**CONTENT**
0	Command echo
1	Sequence number
2	Result code
3	Axis state
4	Fault code
5	Status flags
6-7	Reserved

The sequence number is used to associate responses with transmitted commands. This simplifies synchronization between the master and slave nodes and allows detection of missing or delayed responses.

Status information is transmitted using compact bitwise flags. These flags indicate conditions such as:

- axis homed;
- axis busy;
- active fault;
- emergency-stop condition.

Using flags allows multiple status conditions to be transmitted within a single byte while keeping the CAN payload compact.

All command and response frames were intentionally designed to fit within a single CAN frame. This simplifies firmware implementation and reduces communication overhead on the bus.

The communication protocol was implemented as a separate software layer above the low-level ECAN driver functions. This separation allows the application firmware to operate independently from hardware-specific communication details.

The protocol layer is responsible for:

- encoding outgoing frames;

- decoding received messages;
- validating identifiers and frame length;
- converting data between byte arrays and application variables.

This structure simplified debugging during development because communication handling and axis-control logic could be tested independently.

4. Motion Control Firmware Design and Development

4.1. Development environment and hardware platform

4.1.1. Development Environment

Firmware development was performed using the MPLAB X Integrated Development Environment (IDE) together with the XC8 compiler provided by Microchip Technology.

Peripheral initialization and low-level driver generation were assisted through the MPLAB Code Configurator (MCC), which was used to configure:

- ECAN communication;
- UART communication;
- timers;
- PWM peripherals;
- interrupt configuration;
- input/output pin mapping.

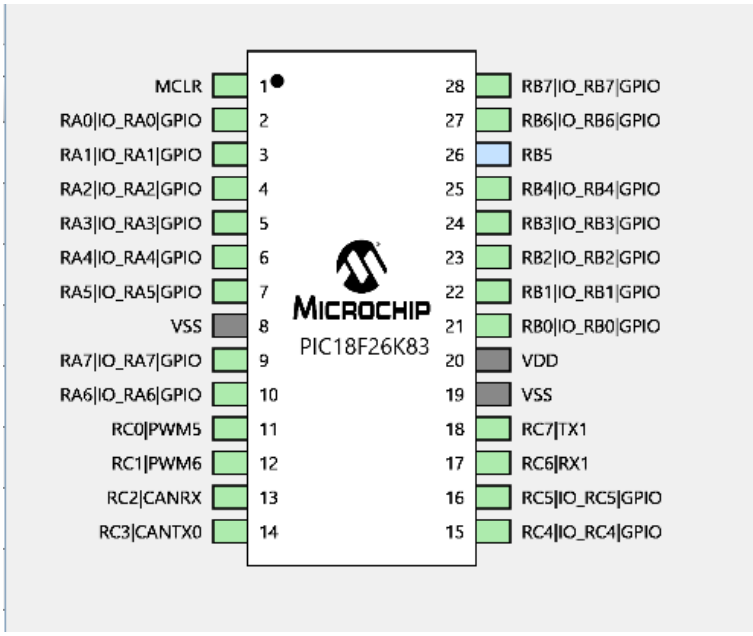

Figure 8 MCC pin manager configuration for the following implementation on the PIC18F26K83

The firmware itself was developed using a modular structure consisting of separate software layers responsible for:

- axis-control logic;
- motor control;
- encoder processing;

- communication handling;
- master/slave coordination.

UART-based diagnostic output was used during development and validation in order to monitor system behavior and simplify debugging during hardware integration.

4.1.2. PIC18F26K83 Microcontroller

The implemented system is based on the PIC18F26K83 microcontroller manufactured by Microchip Technology. [10]

The selected microcontroller provides several hardware features relevant to distributed motion-control applications, including:

- integrated ECAN communication;
- multiple timer peripherals;
- PWM generation modules;
- interrupt support;
- UART communication interfaces;
- sufficient digital input/output capability for encoder and motor interfacing.

The integrated ECAN peripheral simplifies implementation of distributed communication between controller nodes, while the available timers and PWM peripherals support motor-control signal generation and periodic firmware execution.

4.1.3. Hardware Architecture Overview

Table below summarizes the main hardware components:

Table 3 Main hardware components and power supply

Component	Description
PIC18F26K83	Main microcontroller
TB6612FNG	Dual H-bridge motor driver
AS5047P	Magnetic rotary encoder
MCP2551	CAN transceiver
UMOT 28HS2415	Stepper motor
Metrix AX 322	Laboratory power supply

The slave controller integrates the microcontroller, motor driver, encoder interface and CAN communication circuitry on a single printed circuit board.

Motor actuation is performed through the **TB6612FNG** dual H-bridge driver. The driver is controlled by the microcontroller and generates the excitation states required for operation of the bipolar stepper motor.

Position feedback is obtained using the **AS5047P** magnetic rotary encoder. The encoder communicates with the microcontroller through the SPI interface and provides position information used for movement supervision, homing procedures and position tracking.

Communication between controller nodes is implemented through the **MCP2551** CAN transceiver. The transceiver converts the logic-level CAN signals generated by the microcontroller into differential CAN bus signals suitable for communication between distributed nodes.

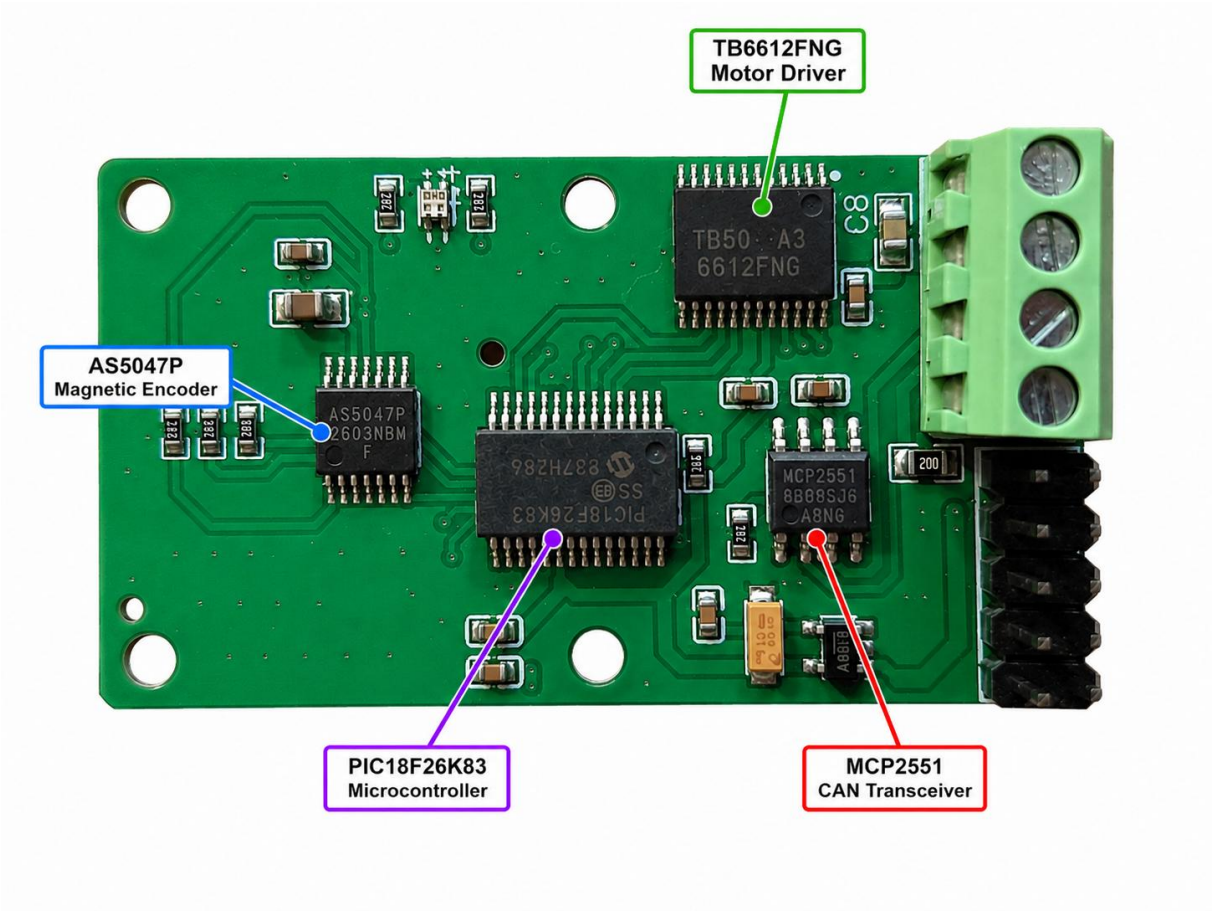

Figure 9 Controller PCB

During validation, the controller boards were connected through a CAN bus and powered from a laboratory PSU operating at approximately 7.0 V. The UMOT 28HS stepper motor was connected directly to the slave node and used for validation of the implemented motion-control algorithms.

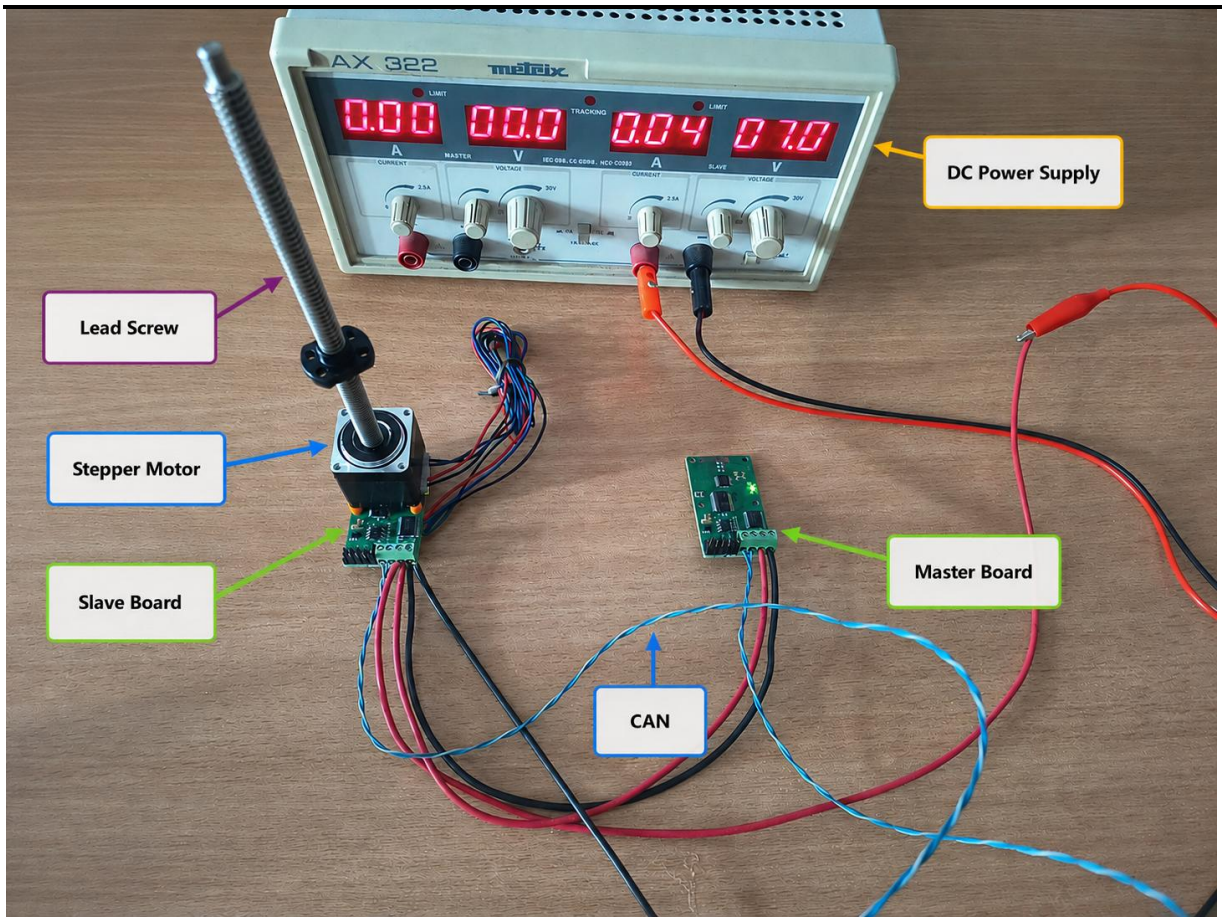

Figure 10 Validation setup consisting of master and slave nodes connected through the CAN bus

The hardware platform provided the necessary processing, communication and feedback capabilities required for development and validation of the distributed motion-control architecture presented in this thesis.

4.2. System Architecture Overview

The developed motion control system is based on a distributed embedded architecture, where motion execution and communication responsibilities are separated across dedicated control nodes. The system consists of a central master controller and one or more axis nodes, each responsible for controlling a single axis of motion on each side.

The master controller acts as the central coordination unit of the system. It is intended to receive commands through a serial communication interface (UART) and translate them into control messages transmitted over a Controller Area Network (CAN) bus. Separating high-level command processing from low-level actuator control reduces timing dependency between communication tasks and motion generation, which simplifies the implementation of predictable control behavior on resource-constrained microcontrollers.

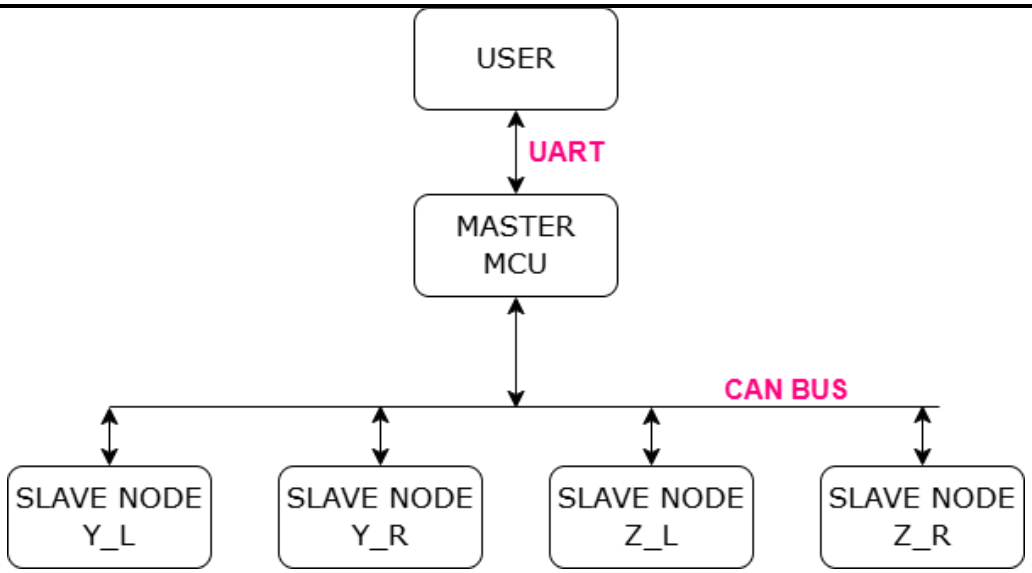

Figure 11 Overall architecture of the subsystems

Each slave node operates as an autonomous axis controller responsible for executing all time-critical motion control operations locally and managing all low-level control functions required for axis operation. These functions include step generation, encoder feedback processing, homing-state execution and local fault supervision. The separation of responsibilities between master and slave nodes reduces system complexity and improves reliability by isolating real-time control from communication coordination.

The internal architecture of a slave node is organized into several functional modules:

- Axis control module - implements the main control logic and state machine governing axis behavior;
- Motor control module - provides an abstraction layer for stepper motor actuation;
- Encoder module - processes position feedback and converts raw counts into physical units, detects motions and stopping conditions for enhanced reliability;
- Configuration management module - stores and loads calibration parameters from the internal EEPROM memory, provides access to persistent configuration values and ensures that system settings are restored automatically after power-up;
- Communication interface - decodes CAN command frames, dispatches axis operations and generates status responses.

The axis control module serves as the central coordinator, integrating all other modules into a centralized control structure. The axis module abstracts the underlying motor and encoder implementations and exposes a unified control interface to higher-level firmware layers. This

separation allows changes in hardware-specific functionality without affecting the main control logic.

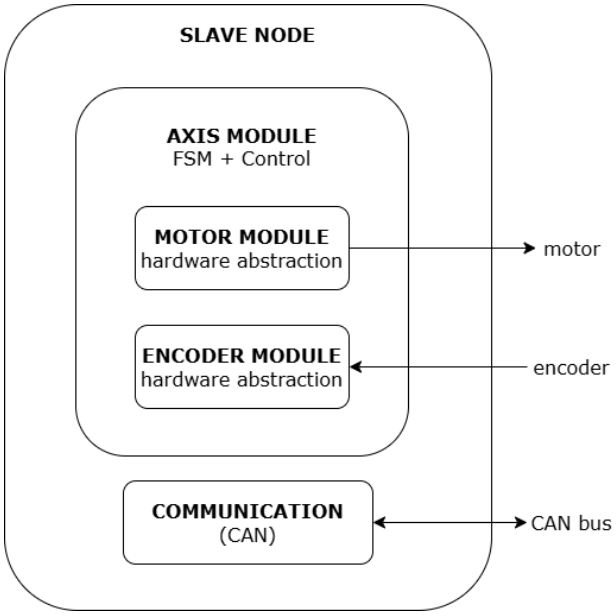

Figure 12 Internal architecture of a slave node

Communication between the master and slave nodes follows a command-based protocol. High-level commands such as homing, movement and emergency stop are transmitted over the CAN bus and interpreted by the slave node. These commands are directly mapped to axis control functions within the firmware, ensuring a structured command-execution workflow.

Besides motion-control operations, the command-dispatch layer also supports configuration management. Configuration commands can update calibration parameters stored in the internal EEPROM memory, allowing persistent system settings to be modified without changing the application firmware.

4.3. Axis Control Module

The axis control module represents central control layer of the slave firmware architecture. It is responsible for coordinating all control operations associated with a single mechanical axis, including motion execution, homing procedure, feedback processing and safety handling. The module integrates multiple subsystems into a unified control structure and serves as the central coordinator of axis behavior. The design of the axis module is based on a structured data model combined with a periodic execution function.

4.3.1. Axis Data Structure

The axis is represented by a dedicated data structure that encapsulates all information required for motion execution, position tracking and fault supervision. The structure stores the

current operational state of the axis together with the target position, homing-related variables and references to the associated motor and encoder modules.

```
typedef struct {
    AXIS_State_t state;
    AXIS_Fault_t faultCode;

    float currentPositionMm;
    float targetPositionMm;
    float positionDeltaMm;

    bool homed;
    bool emergencyActive;

    AXIS_HomingStep_t homingStep;
    uint32_t homingStartMs;
    uint8_t homingDetectionCount;

    float homingReleaseTargetMm;
    bool homingReleaseMotionVerified;
    float homingSeekStartPosMm;
    bool homingSeekMotionVerified;

    int32_t homingLastRawCount;
    uint32_t homingLastMotionMs;
    bool homingStallDetectionArmed;

    MOTOR_t *motor;
    ENCODER_t *encoder;

    AXIS_Config_t config;
} AXIS_t;
```

Listing 2 AXIS_t Struct

The configuration sub-structure contains parameters loaded during system initialization, including travel limits, movement speeds and homing settings. Separating these parameters from the control logic simplifies adaptation of the firmware to different hardware configurations and calibration values.

In addition to configuration data, the structure maintains runtime variables required by the homing procedure. These include motion-verification flags, reference-detection counters, timeout tracking variables and encoder-based stall-detection information. Storing these variables within the axis object allows the homing and motion-control state machines to operate using a single, self-contained software entity.

4.3.2. Periodic Task Execution Model

The axis control logic operates within a periodic execution model, ensuring that feedback processing, state evaluation and motion decisions are updated during axis movement.

The function AXIS_Task(AXIS_t *axis) (see *Appendix A*) performs the following operations during each execution cycle:

1) Feedback update - encoder data is processed to determine the current position;

2) Stop condition check - raw count data is evaluated to detect motion or stop conditions;

3) Safety checks - fault and emergency conditions are verified;

4) State machine execution - control logic is executed based on the current state.

4.3.3. State-Based Control Approach

The axis module is implemented as a finite state machine (FSM), where each state corresponds to a specific operational mode of the axis. This approach simplifies implementation of complex control behavior by separating motion execution, homing and fault handling into explicitly defined operational states. A more detailed explanation of the implementation of the FSM is provided in section 4.3.

The primary states are:

- NOT_HOMED - initial condition where movement is restricted;
- HOMING - execution of the reference detection procedure;
- READY - idle state, awaiting commands;
- MOVING - active motion toward a target position;
- STOPPING - controlled deceleration and halt;
- EMERGENCY - immediate stop condition;
- FAULT - error state requiring system reset.

The use of FSM ensures controlled transitions between operating modes and improves system safety and maintainability.

4.3.4. Integration of Subsystem Modules

The axis module integrates several subsystems, each responsible for a specific function:

- The motor module executes physical motion by generating step signals;
- The encoder module provides position feedback in physical units, detects motion state and supports homing logic;

These subsystems are integrated through abstraction-layer interfaces rather than direct hardware access, allowing the axis control logic to remain independent of hardware specific implementation details.

As a result modifications to low-level actuator or feedback implementations can be introduced without requiring structural changes to the axis control logic. For example, changes in the step generation method or hardware driver configuration can be implemented within the motor module, while the axis control logic remains unchanged.

This modular approach improves maintainability and allows individual components to be developed and tested independently.

4.3.5. Configuration and Flexibility

A key characteristic of the axis module is the separation between control logic and configurable operational parameters. All important operational parameters are defined within a configuration structure:

- motion speed settings;
- travel limits;
- homing behavior parameters;
- directional conventions.

```
typedef struct {
    float minTravelMm;
    float maxTravelMm;
    float stepsPerMm;
    float positionToleranceMm;
    float fastZoneMm;

    uint16_t moveSpeedFastHz;
    uint16_t moveSpeedSlowHz;
    uint16_t homingSeekSpeedHz;
    uint16_t homingReleaseSpeedHz;
    uint16_t homingBackoffSpeedHz;

    float homingVerifyDistanceMm;
    float homingFinalBackoffMm;
    float homingMinSeekTravelMm;
    uint8_t homingRequiredDetections;
    uint32_t homingTimeoutMs;

    MOTOR_Direction_t positiveMoveDirection;
    MOTOR_Direction_t homeSeekDirection;

} AXIS_Config_t;
```

Listing 3 AXIS_Config_t struct

The configuration structure allows motion behavior, homing sensitivity and directional conventions to be adjusted independently from the control implementation. This proved particularly useful during experimental validation, where multiple parameter values required tuning. The separation between configuration and control logic allowed calibration values to be adjusted independently from the motion-control implementation.

This parameter-driven approach improves firmware portability and simplifies adaptation of the control system to different mechanical configurations and validation scenarios. In the final implementation, selected configuration parameters can additionally be stored in EEPROM memory, allowing calibration settings to persist across power cycles.

4.4. State Machine Implementation

The operational behavior of the axis controller is implemented using a finite state machine (FSM) integrated within the periodic control task. The FSM approach allows motion execution, homing behavior and fault handling to be represented as explicitly controlled operational states with explicit transitions.

The state-machine architecture improves predictability of system behavior and simplifies implementation of safety-critical transitions such as emergency stopping, fault handling and homing-state progression. Each state represents a distinct mode of operation, while transitions between states are governed by explicitly defined conditions.

The implementation is based on a centralized switch-case dispatcher executed within the periodic axis task. This structure keeps all state transitions within a single execution context and avoids fragmented control behavior.

4.4.1. State Definition

The axis control module defines a set of operational states that describe the complete lifecycle of the axis:

- NOT_HOMED - initial condition where movement is restricted;
- HOMING - execution of the reference detection procedure;
- READY - idle state, awaiting commands;
- MOVING - active motion toward a target position;
- STOPPING - controlled deceleration and halt;
- EMERGENCY - immediate stop condition;
- FAULT - error state requiring system reset.

Each state corresponds to a specific control strategy and defines how motion commands, feedback data and safety conditions are processed during execution.

The state machine is executed within the periodic axis task function. During each execution cycle the system evaluates the current state and executes the corresponding control logic.

The implementation is based on a structured switch-case construct, ensuring that:

- only one state is active at any time;
- state transitions are explicit and controlled;
- system behavior remains consistent across cycles.

4.4.2. State Transitions

Transitions between states are triggered by specific conditions, including user commands, encoder feedback and fault detection:

- NOT_HOMED → HOMING

  Triggered by a homing command from the master controller;
- HOMING → READY

  Occurs after successful completion of the homing sequence;
- READY → MOVING

  Initiated when a valid movement command is received;
- MOVING → READY

  Triggered when the target position is reached within tolerance;
- MOVING → STOPPING

  Occurs when a stop command is issued;
- ANY STATE → EMERGENCY

  Triggered by an emergency stop condition;
- ANY STATE → FAULT

  Occurs when a critical error is detected.

Transition conditions are evaluated during execution of the periodic control task. This allows the system to react during the next control-cycle execution to feedback changes, command requests or abnormal operating conditions.

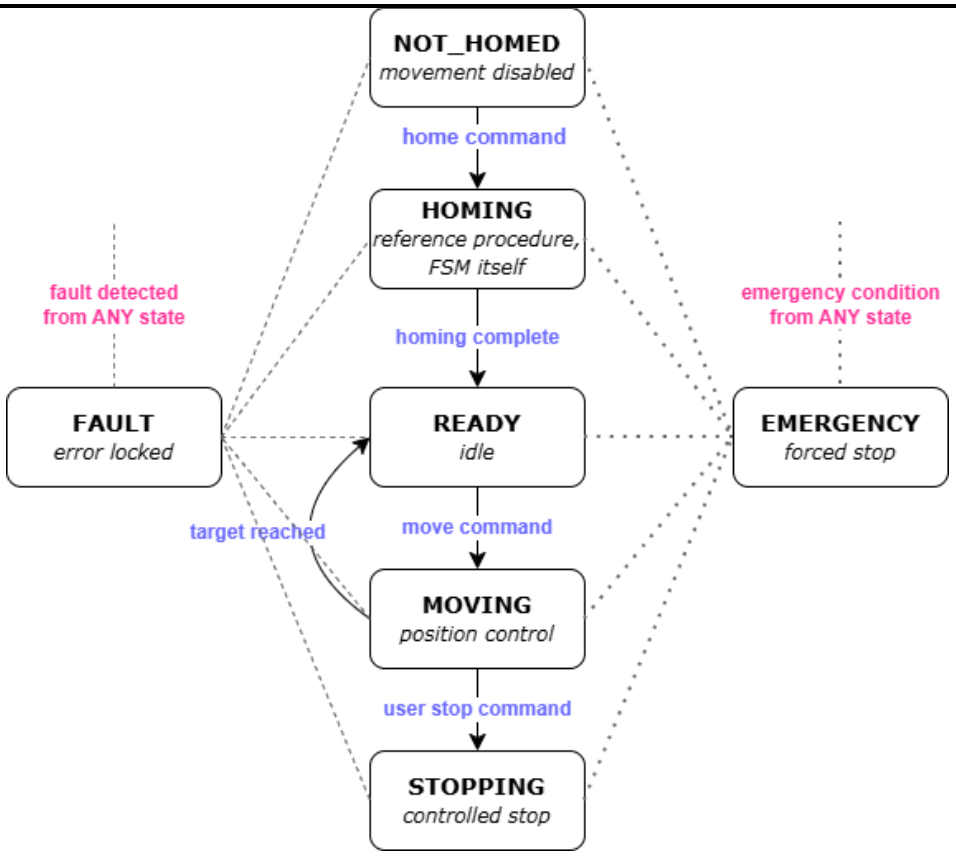

Figure 13 FSM representation of the AXIS logic

4.4.3. Motion State Behavior

Motion control is based on evaluation of the position error, defined as the difference between the target position and current position.

```
axis->positionDeltaMm = axis->targetPositionMm - axis->currentPositionMm;
```

Listing 4 Position error calculation

The position error is calculated and used to determine both direction and speed of motion. The direction is determined by the sign of the position error and is translated into signals through the motor module.

Motion is terminated when the position error falls within a predefined tolerance range, ensuring accurate positioning while *avoiding oscillatory behavior* near the target, as demonstrated with the following snippet contained in the state AXIS_STATE_MOVING.

```
…
if(absDeltaMm <= axis->config.positionToleranceMm) {
                AXIS_StopMotor(axis);
                axis->positionDeltaMm = 0.0f;
                axis->state = AXIS_STATE_READY;
                break;
            }
…
```

Listing 5 Snippet of AXIS_STATE_MOVING

This tolerance-based stopping strategy compensates for minor feedback fluctuations and prevents repeated state switching near the target position, which demonstrates itself as oscillatory behavior, after that the system transitions to the **READY** state.

4.4.4. Homing State Behavior

The **HOMING** state internally operates as a secondary finite state machine embedded within the primary axis FSM, implemented as a secondary state machine within the main FSM. This nested structure allows the homing procedure to be broken down into manageable stages.

The homing procedure is handled through a dedicated set of states executed within the main control loop:

- searching for the reference position;
- releasing from the detected reference;
- re-approaching for verification;
- setting the encoder zero position;
- moving to a safe offset position.

Each homing stage is executed as an independent operational phase with dedicated transition conditions and fault checks. This structure improves robustness of the reference-detection process and simplifies verification of homing behavior during testing. Refer to section 4.5 for a detailed breakdown of the homing procedure and state machine.

4.4.5. Fault and Emergency Handling

The FSM includes dedicated states for fault and emergency conditions ensuring safe system behavior under abnormal conditions.

In the event of a fault:

- the motor is immediately terminated;
- the system transitions to the FAULT state;
- further motion commands are blocked until a reset is performed

Similarly, the **EMERGENCY** state provides an immediate and unconditional stop mechanism, overriding all ongoing operations.

4.4.6. Advantages of the State Machine Approach

The use of a finite state machine provides several advantages:

- clear separation of operating modes;

- simplified implementation of complex behaviors;
- improved readability and maintainability of the code;
- easier debugging and testing;
- predictable system response.

4.5. Homing algorithm and Reference Detection

The homing procedure represents one of the most important operations within the motion-control system, as it establishes the reference coordinate system used for all subsequent positioning operations.

In the developed firmware, the homing procedure is implemented as a multi-stage verification sequence designed to improve consistency of reference detection and reduce the probability of false reference detection. The approach is based on detecting a mechanical constraint using the encoder reading and verifying the detected position through repeated measurements.

4.5.1. Homing Objectives and Challenges

The primary objective of the homing procedure is to determine a consistent and repeatable reference position without requiring modification of the mechanical structure. This requirement significantly influenced the firmware design, since the system could not rely on permanently mounted reference switches integrated into the frame structure.

A key design decision in this work is the avoidance of traditional mechanical limit switches (endstops). While such switches are commonly used in motion systems, such switches introduce additional challenges, particularly related to signal debouncing. Debouncing requires additional filtering logic and can introduce uncertainty in the precise detection of the switching event, especially in high-precision applications. Instead the implemented approach relies on encoder-based motion supervision to detect loss of mechanical movement when the axis reaches a physical constraint. The firmware evaluates the absence of encoder movement over time while motion commands are still active. This behavior is interpreted as contact with the mechanical limit.

4.5.2. Homing Strategy Overview

The homing algorithm is implemented as a sequence of stages, each designed to improve the reliability of reference detection:

1) Reference search

2) Release from reference

3) Reapproach for verification

4) Encoder zeroing

5) Final backoff

The staged structure allows the reference position to be verified incrementally rather than relying on a single detection event.

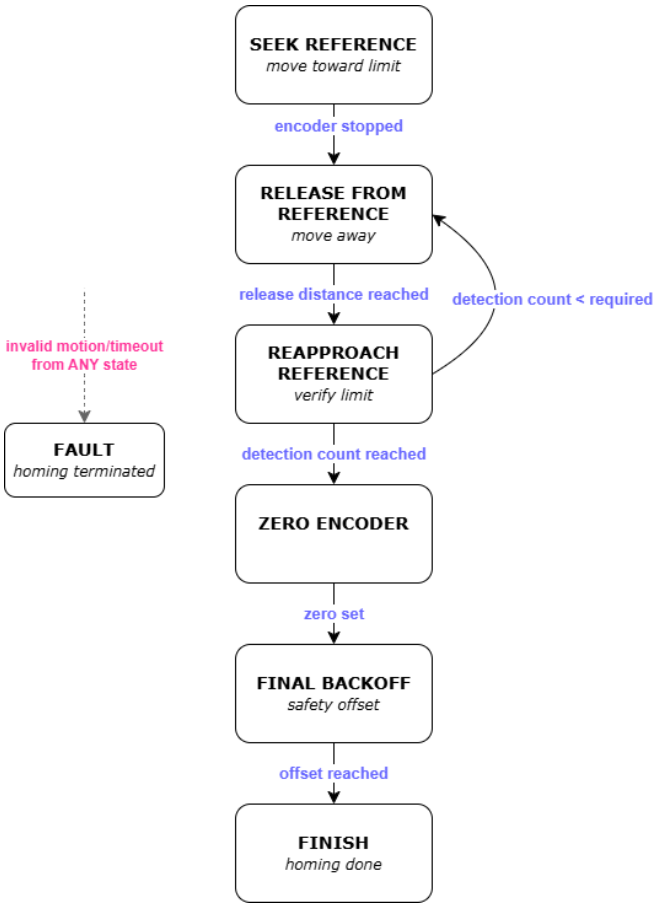

Figure 14 FSM representation of the homing sequence

The homing procedure is implemented as a secondary state machine executed within the primary axis control loop. Each homing stage contains dedicated transition conditions, timeout supervision and motion-validation checks.

4.5.3. Reference Search Phase

During the initial search phase the axis moves towards the expected reference position using a predefined homing speed and directional configuration. The motor is driven at a controlled speed while the encoder monitors shaft movement.

When the encoder feedback indicates that shaft movement has stopped despite continued motor excitation, i.e. when raw count has not changed, this is interpreted as contact with

mechanical limit of the axis travel range. At this point the motor is halted and the system transitions to the next phase.

To avoid false detection caused by temporary disturbances or insufficient movement, the firmware additionally verifies that a minimum travel distance has been exceeded before accepting the stop condition as a valid reference event.

4.5.4. Release and Reapproach Verification

To ensure that the detected reference position is valid and not caused by transient mechanical effects or temporary motion interruption, a verification process is performed.

First the axis moves away from the detected reference position by a predefined distance. This ensures that the mechanical constraint is fully released. The system then performs a second approach toward the reference position.

If the stop condition is detected again, the system increases confidence in the validity of the reference. This process may be repeated multiple times, depending on configuration parameters, to further improve confidence in reference detection.

A detection counter is used to track the number of successful confirmations, ensuring that only consistent detections are accepted as valid.

4.5.5. Encoder Zeroing

After successful verification of the reference position, the encoder is reset to define the zero position of the axis.

The encoder zero is set by storing the current raw count as an offset, allowing all subsequent position calculations to be relative to the detected reference.

4.5.6. Final Backoff

Following encoder zeroing, the axis is moved away from the reference position by a predefined offset distance from the mechanical reference point.

This ensures that the system does not remain in contact with the mechanical limit and allows safe operation during subsequent movement commands.

4.5.7. Safety and Fault Handling During Homing

To ensure safe operation, several fault conditions are monitored during the homing process:

- timeout of the homing sequence;
- failure to detect motion or release;

- invalid movement range;

If any of these conditions occur, the system stops the motor and transitions to the **FAULT** state. Timeout supervision is integrated into the homing state machine to prevent indefinite motion in case of mechanical blockage, encoder malfunction or unexpected system behavior.

The proposed homing strategy provides several advantages:

- eliminates the need for mechanical endstop switches and associated debouncing signal-conditioning logic;
- reduces hardware complexity and potential failure points;
- improves reliability through multi-stage verification;
- ensures repeatable and accurate reference positioning.

Additionally, the use of the encoder enables detection of mechanical constraints without modifying the existing structure, aligning with the non-intrusive design objective of the system.

4.6. Feedback and Position Estimation System

The developed motion control system employs an encoder-based feedback approach for both position estimation and motion state evaluation. This approach improves motion supervision capability while avoiding additional dedicated sensing hardware.

4.6.1. Role of Feedback in the System

Conventional stepper-motor control typically operates under the assumption that all commanded motion is executed correctly. In practical systems, however, this assumption may become invalid under conditions such as excessive load, mechanical blockage or transient dynamic effects.

To address these limitations, feedback mechanisms are introduced to:

- verify actual motion of the axis;
- detect abnormal conditions such as stalls;
- improve reliability of homing procedures;
- provide accurate position estimation.

Although the implemented system does not perform continuous closed-loop position correction, encoder feedback introduces a supervisory verification layer that improves robustness of motion execution and reference detection.

4.6.2. Encoder Signal Processing

The axis-control layer does not read encoder input pins directly, it interacts with the encoder through functions such as `ENCODER_Update()`, `ENCODER_GetRawCount()`, `ENCODER_GetPositionMm()` and `ENCODER_SetZero()`. This allows the axis module to operate using physical position values in millimeters rather than raw hardware signals.

The encoder module is responsible for converting the electrical signals generated by the incremental encoder into position information that can be used by the motion-control system. Two signal-processing approaches were implemented during development and validation.

The first approach uses the rising edge of encoder channel A as the counting event. Whenever a rising edge is detected, the logic level of channel B is sampled to determine the direction of movement. Depending on the state of channel B, the encoder count is either incremented or decremented. This method requires processing fewer signal transitions and provides a relatively simple implementation while still supporting bidirectional position tracking:

```
void ENCODER_ISR_Update(void) {

    uint8_t a;
    a = IO_RB0_GetValue();
    uint8_t b;
    b = IO_RB1_GetValue();

    if (!g_encoderPrevABValid) {
        g_encoderPrevAB = a;
        g_encoderPrevABValid = true;
        return;
    }
    // count rising edge of A, B only for direction
    if ((g_encoderPrevAB == 0u) && (a == 1u)) {
        if (b == 0u)  g_encoderRawCount++;
        else g_encoderRawCount--;
    }
    g_encoderPrevAB = a;
}
```

Listing 6 Single-channel encoder processing using channel A interrupts

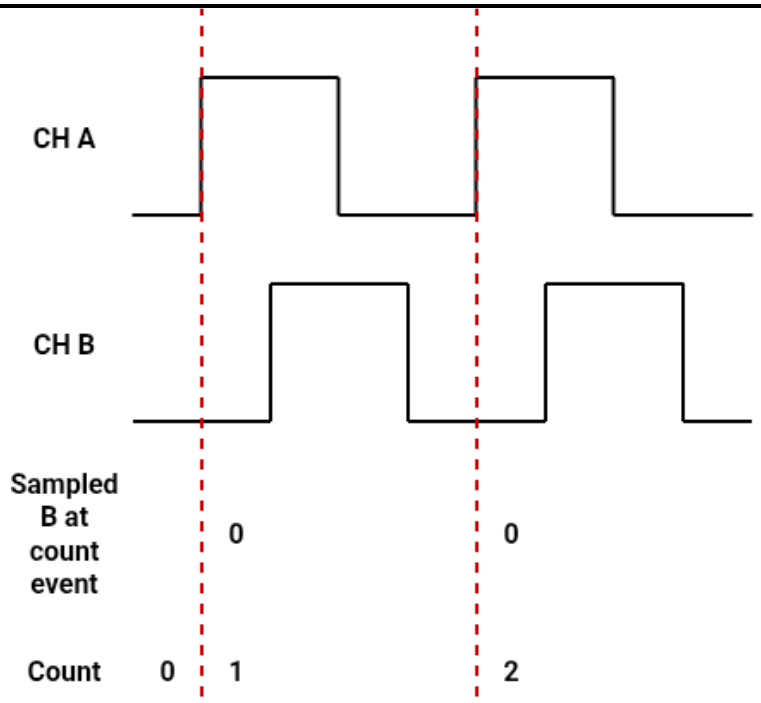

Figure 15 Single-channel encoder counting using channel A rising edges during positive motion

Figure above illustrates the single-channel counting method. Position increments are generated only on the rising edge of channel A. At each count event, the logic level of channel B is sampled to determine the direction of motion. Since channel B is low during the illustrated transitions, the accumulated encoder count increases.

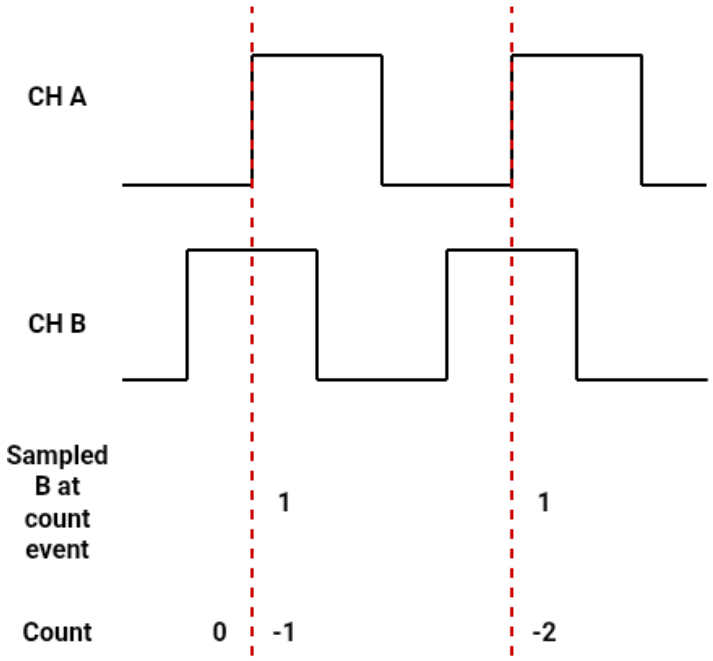

Figure 16 Single-channel encoder counting using channel A rising edges during negative motion

The rising edge of channel A remains the counting event, however channel B is sampled high, causing the accumulated count value to decrease. This allows bidirectional position tracking while requiring only a single interrupt source.

The second approach implements quadrature decoding using both encoder channels A and B. In this mode, the firmware evaluates the complete sequence of state transitions generated by the two encoder signals. The direction of movement is determined from the order of the transitions, while each valid state change contributes to the accumulated encoder count. Compared to the first approach, quadrature decoding provides higher counting resolution and improved motion tracking accuracy.

```
void ENCODER_ISR_A(void) {
    if (IO_RB0_GetValue()) {
        if (IO_RB1_GetValue()) {
            --g_encoderRawCount;
        } else {
            ++g_encoderRawCount;
        }
    } else {
        if (IO_RB1_GetValue()) {
            ++g_encoderRawCount;
        } else {
            --g_encoderRawCount;
        }
    }
}

void ENCODER_ISR_B(void) {
    if (IO_RB1_GetValue()) {
        if (IO_RB0_GetValue()) {
            ++g_encoderRawCount;
        } else {
            --g_encoderRawCount;
        }
    } else {
        if (IO_RB0_GetValue()) {
            --g_encoderRawCount;
        } else {
            ++g_encoderRawCount;
        }
    }
}
```

Listing 7 Quadrature encoder processing using channel A and B interrupts

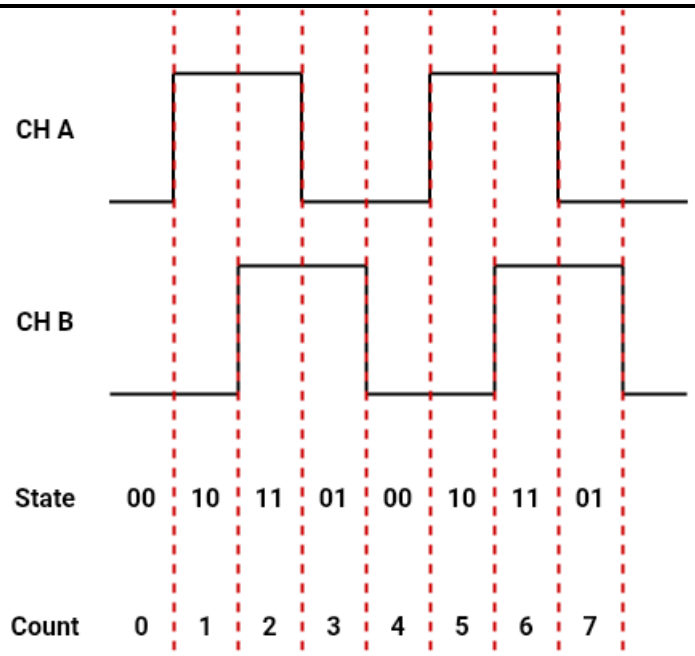

Figure 17 Quadrature encoder state transitions during positive motion

Forward quadrature decoding using both encoder channels. The firmware evaluates the complete sequence of valid encoder states (00 → 10 → 11 → 01 → 00). Each valid transition contributes to the accumulated encoder count, resulting in higher counting resolution compared to the single-channel approach.

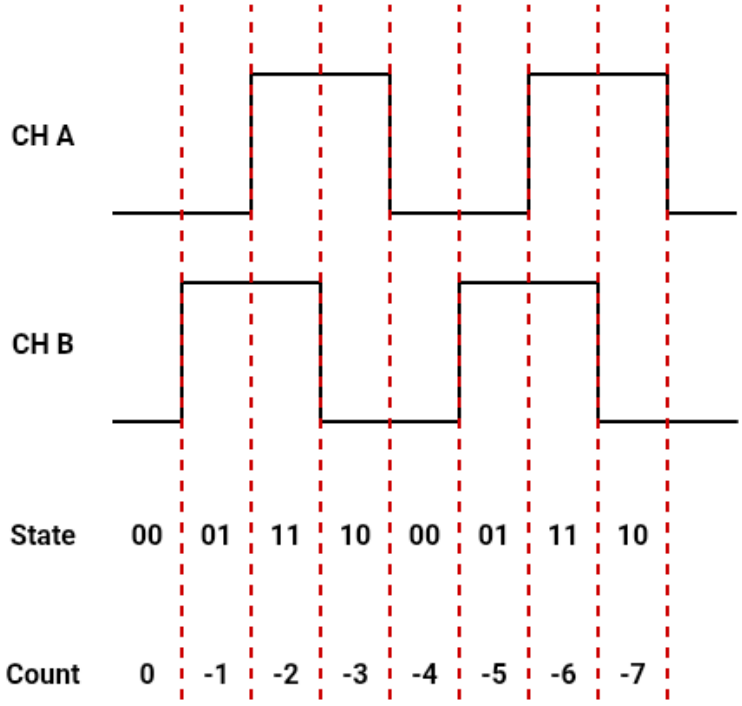

Figure 18 Quadrature encoder state transitions during negative motion

Reversing the order of the state transitions causes the encoder count to change in the opposite direction. By evaluating all signal transitions, quadrature decoding provides improved motion tracking accuracy and increased effective resolution.

In both implementations, A only and Quadrature, the encoder module maintains an accumulated raw-count value representing the relative displacement of the axis. Access to the raw-count value is protected by temporarily disabling global interrupts during read operations, preventing inconsistent values while the interrupt routine may be modifying the count.

```
INTERRUPT_GlobalInterruptHighDisable();
*rawCount = g_encoderRawCount;
INTERRUPT_GlobalInterruptHighEnable();
```

Listing 8 Protected raw-count read operation in encoder.c

The resulting raw-count value forms the basis for all higher-level feedback functions implemented in the system, including position estimation, motion verification and homing-related stall detection. By isolating the low-level signal-processing logic within a dedicated module, the remaining firmware can operate independently of the specific encoder-decoding method being used.

4.6.3. Encoder-Based Position Feedback

The encoder module provides periodic position updates during execution of the control task by converting raw counts into physical displacement units.

Position estimation is performed by converting encoder counts into physical displacement relative to a stored software-defined zero reference. This approach enables the system to:

- track absolute position relative to a defined reference;
- perform accurate positioning operations;
- update position continuously during motion.

```
static float ENCODER_ConvertRawCountToPositionMm (const ENCODER_t *en-
coder, int32_t rawCount) {
    if((encoder == 0) || !ENCODER_IsScaleValid(encoder)) return 0.0f;
    return ((float)rawCount) * encoder->scaleMmPerCount;
}
```

Listing 9 Helper function in encoder.c to convert raw counts to mm

The conversion process isolates low-level encoder representation from higher-level control logic, allowing the axis controller to operate directly with physical units expressed in millimeters. Encoder feedback is updated continuously during execution of the periodic control loop, and position is computed using a scaling factor derived from the mechanical transmission ratio. This ensures consistent mapping between encoder counts and physical displacement.

A zero-offset value is used to define the reference position of the axis. During homing, the axis-control module calls `ENCODER_SetZero()` after the reference position has been detected. This stores the current raw encoder count as the zero reference, allowing subsequent position readings to be expressed relative to the homed coordinate system.

The encoder module also provides validity checking. During each call to `ENCODER_Update()`, the module verifies that the encoder object is initialized, that the scale factor is valid and that the raw count can be read successfully. If one of these checks fails, the encoder is marked invalid and the axis-control layer can react by entering a fault condition.

4.6.4. Encoder-Based Motion Detection

In addition to position tracking, the encoder is used to determine whether the motor shaft is actively moving or has stopped.

The function compares the current encoder count with the last recorded count during homing. If the difference exceeds a minimum threshold, motion is considered present and the last-motion timestamp is updated. If no sufficient encoder movement is detected for a predefined timeout period, the axis is considered stopped. This allows the system to detect contact with a mechanical constraint using only encoder feedback.

This functionality is particularly important for:

- detecting contact with mechanical limits;
- identifying stalled motion;
- supporting reliable homing operations.

```
static bool AXIS_IsEncoderMotionStopped(AXIS_t *axis) {
    int32_t currentRaw;
    int32_t deltaRaw;

    if((axis == 0) || (axis->encoder == 0)) return false;

    currentRaw = ENCODER_GetRawCount(axis->encoder);
    deltaRaw = currentRaw - axis->homingLastRawCount;
    if (deltaRaw < 0) deltaRaw = -deltaRaw;

    if (deltaRaw >= AXIS_HOMING_STALL_MIN_COUNTS) {
        axis->homingLastRawCount = currentRaw;
        axis->homingLastMotionMs = millis();
    }

    if ((millis() - axis->homingLastMotionMs) >=
AXIS_HOMING_STALL_TIMEOUT_MS) return true;
```

```
    return false;
}
```

Listing 10 Helper function in axis.c to detect absence of movement

This approach enables detection of mechanical constraints without requiring dedicated limit sensors or additional motion-detection hardware.

4.6.5. Advantages of the Encoder-Based Feedback Approach

The use of a single encoder for both position estimation and motion detection provides several advantages:

- improved detection of mechanical constraints without dedicated endstop switches;
- enhanced reliability of homing procedures;
- ability to detect stalled or abnormal motion.

This approach aligns with the design objective of minimizing hardware modifications while maintaining accurate and reliable system behavior.

4.7. Motor Module

The motor module provides the low-level actuation interface between the axis-control logic and the stepper motor hardware. Its primary responsibility is to translate high-level motion commands, such as direction and stepping frequency, into the electrical excitation sequence required by the motor driver circuitry.

The axis-control module does not directly manipulate motor output pins. Instead, it commands the motor through interface functions such as `MOTOR_Enable()`, `MOTOR_SetDirection()`, `MOTOR_SetStepFrequencyHz()`, `MOTOR_Start()` and `MOTOR_Stop()`. This keeps the axis state machine independent from the hardware-specific details of motor excitation.

4.7.1. Phase Control and Excitation

The implemented stepper motor is driven using two motor phases, designated as phase A and phase B. Each phase is controlled through a bridge circuit consisting of two polarity-control signals and one enable signal. The polarity-control signals determine the direction of current flow through the winding, while the enable signal activates or disables the corresponding phase.

In the developed implementation, the enable outputs are used to energize the motor phases, while the commutation sequence itself is generated entirely in software. The firmware

therefore controls the magnetic field orientation of the motor by selecting the appropriate combination of phase states.

The motor module defines dedicated helper functions for the possible operating states of each phase. A phase can be energized with positive polarity, energized with reversed polarity or disabled completely.

```
static void MOTOR_PhaseAForward(void) {
    PHASE_A_IN1_ON();
    PHASE_A_IN2_OFF();
    PHASE_A_PWM_ON();
}

static void MOTOR_PhaseAReverse(void) {
    PHASE_A_IN1_OFF();
    PHASE_A_IN2_ON();
    PHASE_A_PWM_ON();
}

static void MOTOR_PhaseAOff(void) {
    PHASE_A_PWM_OFF();
    PHASE_A_IN1_OFF();
    PHASE_A_IN2_OFF();
}
```

Listing 11 Phase-control helper functions in motor.c

The actual stepper commutation sequence is implemented by selecting combinations of phase states. In full-step mode, both phases are energized during each step. The implemented full-step sequence is:

```
static void MOTOR_ApplyFullStep(uint8_t index) {
    switch (index & 0x03u) {
        case 0:
            MOTOR_PhaseAForward();
            MOTOR_PhaseBForward();
            break;

        case 1:
            MOTOR_PhaseAForward();
            MOTOR_PhaseBReverse();
            break;

        case 2:
            MOTOR_PhaseAReverse();
            MOTOR_PhaseBReverse();
            break;

        case 3:
        default:
            MOTOR_PhaseAReverse();
            MOTOR_PhaseBForward();
            break;
    }
```

```
}
```

Listing 12 Full-step excitation sequence in motor.c

This sequence produces rotation by changing the magnetic field orientation of the two motor phases in a cyclic pattern. Reversing the stepping order changes the direction of rotation.

The motor module also supports half-step operation. In this mode, the firmware alternates between single-phase and dual-phase excitation states. This provides additional intermediate rotor positions compared to full-step operation. One can select the operation mode with a compile-time flag.

Table 4 Half-step excitation sequence for the bipolar two-phase stepper motor

Step	**Phase A**	**Phase B**
1	A	B
2	A	0
3	A	$\overline{B}$
4	0	$\overline{B}$
5	$\overline{A}$	$\overline{B}$
6	$\overline{A}$	0
7	$\overline{A}$	B
8	0	B

In the table, A and B denote positive current flow through the corresponding motor winding, while $\overline{A}$ and $\overline{B}$ indicate reversed current direction. A value of 0 means that the respective phase is not energized. The sequence alternates between single-phase and dual-phase excitation states to achieve half-step operation with improved positioning resolution and smoother motion.

The implemented firmware supports both excitation strategies. In the current version, the selected operating mode is defined during compilation.

4.7.2. Step Generation and Speed Control

Motor speed is controlled through the requested step frequency. The function `MOTOR_SetStepFrequencyHz()` stores the desired stepping rate, while the internal step generator converts this frequency into a timer-based stepping interval.

The stepping interval is calculated using the timer interrupt frequency:

```
ticks = ((uint32_t)MOTOR_TIMER_ISR_HZ) / ((uint32_t)frequencyHz);
```

Listing 13 Calculation of timer ticks per motor step

During timer interrupt execution, the module increments an internal counter. When the configured number of timer ticks is reached, the next commutation state is applied by advancing the step index in `MOTOR_TimerISR()`.

When the motor is stopped or disabled, all outputs are deactivated and both motor phases are switched off. This prevents unintended energization of the motor driver and ensures safe behaviour when the system enters an idle, fault or emergency state.

The implemented motor-control structure therefore provides a hardware-specific actuation layer while preserving a clean interface toward the higher-level axis controller. This separation was important during development because the same axis-control logic could be tested with substitute hardware and later adapted toward the final stepper-motor driver configuration with zero changes to the higher-level firmware.

4.8. Modular Firmware Architecture

4.8.1. Modular Structure

The developed firmware is organized as a collection of cooperating software modules, each responsible for a specific aspect of the control system. The primary modules are:

- Axis module - coordinates motion execution, homing procedures and state-machine operation;
- Motor module - provides low-level actuator control and stepping signal generation;
- Encoder module - processes feedback data, converts raw count into physical units and supervises movement behavior;
- Configuration-management module - stores and provides persistent system parameters used during initialization and operation;
- Slave node module - provides the communication-dispatch interface between CAN commands and axis operations.

Each module exposes a clearly defined public interface while encapsulating its internal implementation details. This separation minimizes coupling between subsystems and prevents implementation changes from propagating across the firmware architecture.

The slave-node module operates as an interface layer between communication handling and motion-control execution by mapping validated CAN commands directly to axis operations. This ensures that communication mechanisms remain independent of the underlying control implementation.

4.8.2. Layered Architecture

The firmware follows a layered software architecture consisting of three abstraction levels:

1) Application layer - high-level axis control logic and state machine execution;

2) Abstraction layer - motor, encoder and communication interfaces;

3) Hardware layer - microcontroller peripherals and low-level platform-specific drivers.

The application layer operates exclusively through abstraction-layer interfaces rather than direct hardware manipulation, allowing the control logic to remain independent of hardware-specific details. As a result the system can be adapted to different hardware platforms with minimal structural changes.

4.8.3. Encapsulation and Interface Design

Each software module is implemented using a clear separation between public interface functions and internal implementation details. This approach improves readability, reduces unintended interactions between subsystems and simplifies debugging and maintenance activities.

For example, the motor module exposes only a limited set of public control functions, while the implementation details related to commutation sequences, timer handling and output control remain hidden within the module itself.

The listing below shows a subset of the public interface provided by the motor module:

```
void MOTOR_SetDirection(MOTOR_t *motor, MOTOR_Direction_t direction);
void MOTOR_SetStepFrequencyHz(MOTOR_t *motor, uint16_t frequencyHz);
void MOTOR_Start(MOTOR_t *motor);
void MOTOR_Stop(MOTOR_t *motor);
MOTOR_State_t MOTOR_GetState(const MOTOR_t *motor);
```

Listing 14 Snippet from motor.h showing the public functions

The axis-control layer interacts with the motor exclusively through these interface functions, without requiring direct access to the underlying hardware implementation. The same design principle is applied throughout the firmware architecture and forms the basis of the modular software structure.

4.9. Configuration Management and EEPROM Storage

During development and validation of the motion-control firmware, several parameters required periodic adjustment depending on the hardware configuration and testing scenario.

Examples include encoder scaling factors, motion-calibration values and travel-related limits used by the axis-control logic. Initially, these parameters were defined directly in the source code, meaning that every modification required recompilation and reprogramming of the microcontroller.

To improve flexibility, a dedicated configuration-management module was introduced. The module is responsible for storing selected configuration parameters in the internal EEPROM memory of the PIC18F26K83 microcontroller and restoring them automatically during system startup. This allows important calibration data to remain available even after the controller has been powered down.

The configuration-management module operates independently from the motion-control logic. During initialization, the firmware loads the stored configuration from EEPROM memory and makes the parameters available to the corresponding software modules. If no valid configuration is present, a predefined set of default values is loaded instead:

```
AXIS_Initialise(&axis, &motor, &encoder);

    if (!APP_CONFIG_Load(&axis, &motor, &encoder)) {
    APP_CONFIG_Save(&axis, &motor, &encoder);
    }
```

Listing 15 Loading configuration during startup

This ensures predictable system behavior during the first startup and after memory reinitialization.

The stored parameters are primarily related to calibration and motion-control settings. Examples include the conversion between encoder counts and physical displacement, the relationship between motor steps and linear movement, and travel limits used for position supervision.

```
typedef struct {
    uint16_t magic;
    uint8_t version;

    float encoderScaleMmPerCount;
    float maxTravelMm;
    float fastZoneMm;
    float positionToleranceMm;

    uint16_t moveSpeedFastHz;
    uint16_t moveSpeedSlowHz;

    uint16_t checksum;
} APP_CONFIG_Data_t;
```

Listing 16 Configuration structure stored in EEPROM

By storing these values in non-volatile memory the same firmware can be adapted to different hardware configurations without requiring modifications to the application code.

The implemented EEPROM support proved particularly useful during validation. Calibration values could be adjusted and stored permanently without repeatedly modifying firmware source files. This simplified the testing process and reduced the time required to evaluate different configurations.

A possible future extension of the configuration-management module is the storage of additional operating modes and advanced configuration parameters. This would allow a larger portion of the firmware behavior to be modified at runtime without requiring recompilation of the application software.

4.10. Distributed Communication Protocol over CAN

The developed motion-control uses a distributed communication architecture based on a Controller Area Network (CAN) bus. A central master controller exchanges commands and status information with multiple slave nodes responsible for local axis control.

The CAN bus was selected due to its built-in arbitration, error handling and suitability for distributed embedded systems. Its differential signaling also improves communication reliability in electrically noisy environments, which is important for motion-control applications involving motors and switching electronics.

4.10.1. Communication Model

The system follows a command–response communication structure. The master controller generates high-level commands based on user input, while each slave node executes the requested operation locally and returns execution status information.

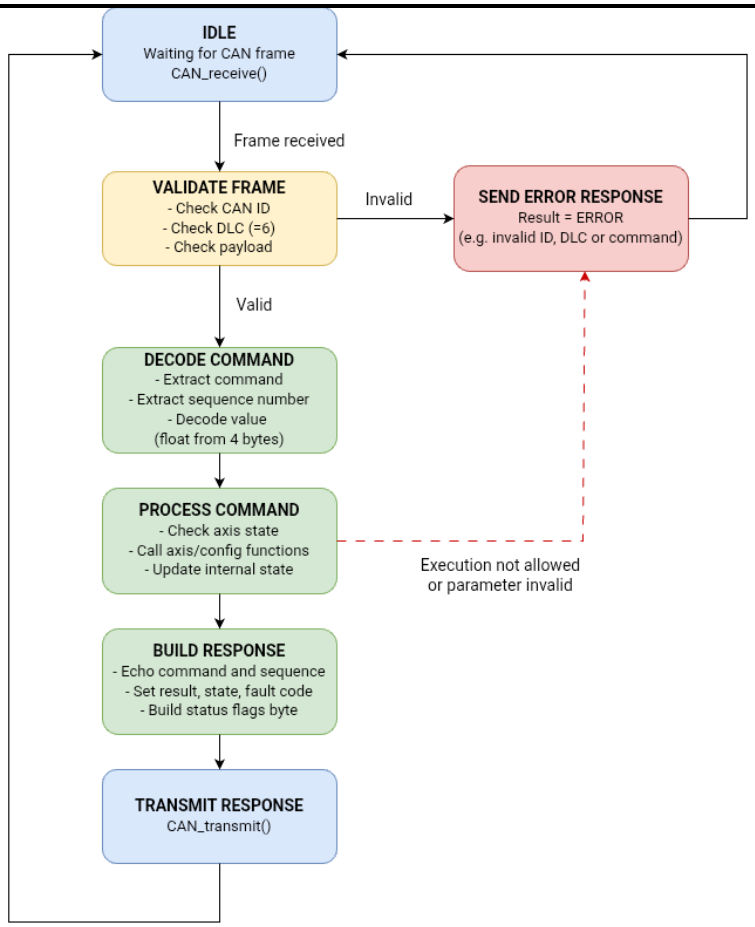

Figure 19 CAN command processing state machine

Presented above - the command-processing sequence executed by a slave node after reception of a CAN frame. Incoming messages are validated and decoded before being forwarded to the application layer. Following command execution, a response frame containing status information is generated and transmitted back to the master node.

Each slave node is responsible for::

- local axis control;
- homing execution;
- motion supervision;
- fault handling;
- encoder processing;
- storage and management of persistent configuration parameters.

The master controller supervises the overall system state and validates commands before transmission.

This separation reduces the processing load of the master controller while allowing time-critical motion behavior to remain local to each axis node.

4.10.2. Message Structure

Communication is implemented using fixed-length CAN frames with a standardized data layout.

Table 5 Command Frame (Master → Slave)

BYTE	**CONTENT**
0	Command identifier
1	Sequence number
2-5	Parameter (float, e.g. position in mm, speed in Hz)
6-7	Reserved

The command identifier defines the requested axis operation, including homing, movement execution or stopping behavior. The sequence number is used to match responses with corresponding requests.

Table 6 Response Frame (Slave → Master)

BYTE	**CONTENT**
0	Command echo
1	Sequence number
2	Result code
3	Axis state
4	Fault code
5	Status flags
6-7	Reserved

The response frame combines command acknowledgment with compact status reporting, allowing the master node to continuously supervise the operational state of each axis.

4.10.3. Command Mapping

Each received command is validated by the slave-node layer and mapped directly to the corresponding axis-control function.

Supported operations include:

- homing initiation;
- absolute movement;
- relative movement;
- controlled stop;
- emergency stop;
- fault reset;
- status requests;
- configuration updates and EEPROM storage operations.

The following example shows command mapping inside the slave-node layer.

```
…
switch (command) {
        case AXIS_NODE_CMD_HOME:
            return AXIS_Home(node->axis);
…
}
…
```

Listing 17 Snippet from slave_node.c, showing mapping to AXIS functions

This structure separates communication handling from low-level axis implementation and simplifies future protocol expansion. In addition to motion-control commands, the protocol also supports configuration-related operations. Configuration values received from the master node can be forwarded to the configuration-management module and stored in EEPROM memory. This allows calibration parameters to be modified through the CAN interface while remaining available after a power cycle.

4.10.4. Node Addressing

Each axis is assigned dedicated CAN identifiers for command reception and status transmission.

Example identifier allocation:

```
#define CAN_ID_MASTER_TO_Y_L 0x101u
#define CAN_ID_MASTER_TO_Y_R 0x102u
#define CAN_ID_MASTER_TO_Z_L 0x103u
#define CAN_ID_MASTER_TO_Z_R 0x104u
```

```
#define CAN_ID_Y_L_TO_MASTER 0x181u
#define CAN_ID_Y_R_TO_MASTER 0x182u
#define CAN_ID_Z_L_TO_MASTER 0x183u
#define CAN_ID_Z_R_TO_MASTER 0x184u
```

Listing 18 CAN identifier definition of each axis from can_protocol.h

This structure simplifies CAN filtering and ensures that each node processes only messages relevant to its assigned axis.

4.10.5. Reliability Mechanisms

Several lightweight supervisory mechanisms are implemented at application level to improve communication reliability.

- sequence-based response matching;
- acknowledgment supervision;
- retry handling;
- identifier and data-length validation.

Each transmitted command contains a sequence number which is echoed by the slave node in its response. This allows the master controller to associate responses with the correct requests and detect invalid or delayed acknowledgments.

If no valid response is received within the configured timeout interval, the master retransmits the command up to a limited retry count.

During integration testing, incorrect sequence values and invalid responses were intentionally injected to validate retransmission behavior and command supervision.

4.10.6. Status Reporting

Each response message includes status flags representing the current condition of the axis:

- homed state;
- busy status;
- fault condition;
- emergency stop status.

This allows the master node to maintain updated representation of axis state, fault conditions and operational availability during runtime.

4.11. Master Node Control Logic

The master node is responsible for system-level supervision within the distributed motion-control architecture. It translates user input into CAN commands, manages communication with slave nodes and maintains an updated representation of the state of each axis.

Unlike the slave nodes, which execute motion-control functions locally, the master focuses on command validation, communication supervision and coordination of multiple nodes.

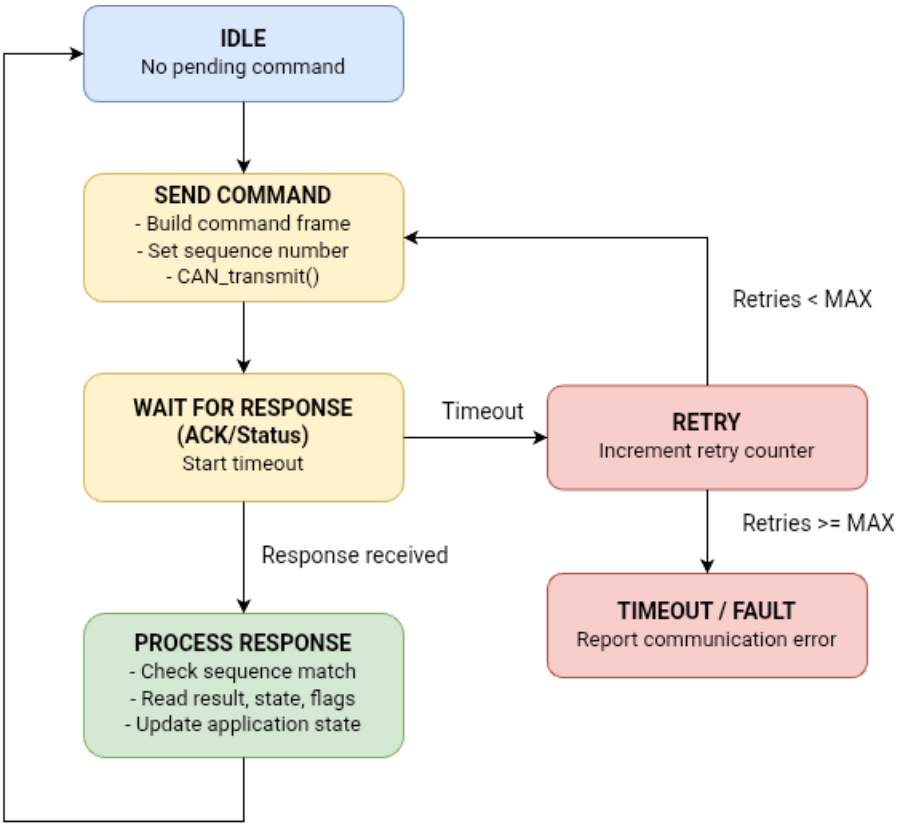

Figure 20 Master node communication state machine

Presented above the command-transmission and acknowledgment-supervision procedure implemented by the master node. After transmitting a command, the master waits for a matching response and verifies the received sequence identifier. Timeout conditions trigger retransmission attempts up to a predefined retry limit, after which a communication fault is reported.

4.11.1. Command Validation

Before transmitting a command, the master verifies that the requested operation is valid for the current axis state. Validation is performed using status information received from the slave nodes.

Commands may be rejected if:

- the axis is busy;

- the axis is in a fault state;
- the axis is in an emergency-stop condition;
- the requested operation is not valid for the current state.

For example, movement commands are rejected while an axis reports a BUSY condition.

```
if(MASTER_AXIS_CLIENT_IsBusy(axisId)) {
            send_string("REJECT MOVE: AXIS BUSY\r\n");
            return false;
        }
```

Listing 19 Rejection logic in axis_client.h utilized by the master MCU

This validation layer prevents invalid commands from being transmitted and reduces unnecessary communication traffic.

4.11.2. Communication Supervision

Each transmitted command contains a sequence identifier used for acknowledgment matching. After transmission, the master waits for a response from the corresponding slave node and verifies that the received message contains the expected identifier and sequence number.

If a valid response is not received within the configured timeout interval, the command may be retransmitted. This mechanism improves communication robustness and helps detect lost or invalid messages.

4.11.3. State Tracking and Coordination

The master maintains a local representation of the state of each slave node, including:

- current axis state;
- fault conditions;
- homing status;
- busy and emergency-stop flags.

This information is updated using status responses received from the slave nodes and is subsequently used during command validation and system supervision.

The implemented architecture keeps time-critical motion execution local to the slave controllers while allowing the master node to coordinate operation of multiple axes through a single communication interface.

4.12. Multi-Axis Coordination and Execution

Each axis is controlled locally by its corresponding slave controller, while the master node supervises communication and overall coordination.

The developed architecture supports distributed control of multiple motion axes through independent slave nodes connected over the CAN bus. Each slave node executes local motion-control functions, while the master controller is responsible for communication and overall system coordination.

The current implementation supports four logical axis nodes:

- Y-axis left (Y_L);
- Y-axis right (Y_R);
- Z-axis left (Z_L);
- Z-axis right (Z_R).

This structure reflects the intended mechanical configuration of the stereotactic positioning system and allows both independent and coordinated axis operation.

Each slave node contains its own motion-control state machine, encoder feedback processing, homing logic and fault supervision mechanisms. Because motion execution is performed locally, communication delays do not directly interrupt an active movement.

Coordinated operation is performed by the master controller. Commands can be issued to multiple axes, while acknowledgments and status responses are monitored to ensure that all involved nodes are in a suitable operating state. In the current implementation, synchronization is achieved through command sequencing and response supervision rather than hardware-level synchronized motion control.

The firmware follows a periodic execution model centered around the **`AXIS_Task()`** routine. During each execution cycle, encoder feedback is updated, motion and fault conditions are evaluated, the state machine is executed and motor-control parameters are updated.

Low-level motor signal generation is handled separately inside the motor module using timer-based execution. This allows the control loop to focus on supervision and decision making without generating individual motor steps directly. As a result, small variations in task execution timing do not significantly affect motor operation.

The modular structure of the firmware allows additional axes to be integrated with minimal modification. Expansion primarily requires the addition of new CAN identifiers and the corresponding master-node command handling, while the existing motion-control architecture remains unchanged.

5. Experimental Validation

5.1. Preliminary Firmware Validation

5.1.1. Setup Using Substitute Hardware

During the early stages of development, full validation on the final stereotactic frame hardware was not continuously possible due to limited availability of the mechanical platform and incomplete integration of all subsystems. For this reason an intermediate validation approach was adopted in order to verify the embedded firmware and motion-control logic before final hardware integration.

The setup consisted of two PIC18F26K83-based controller boards connected through a CAN bus interface, a DC motor and a linear encoder. Although this hardware configuration differs from the final stepper-motor-based actuation concept, it provided a practical platform for validation of the implemented firmware architecture and motion-control algorithms.

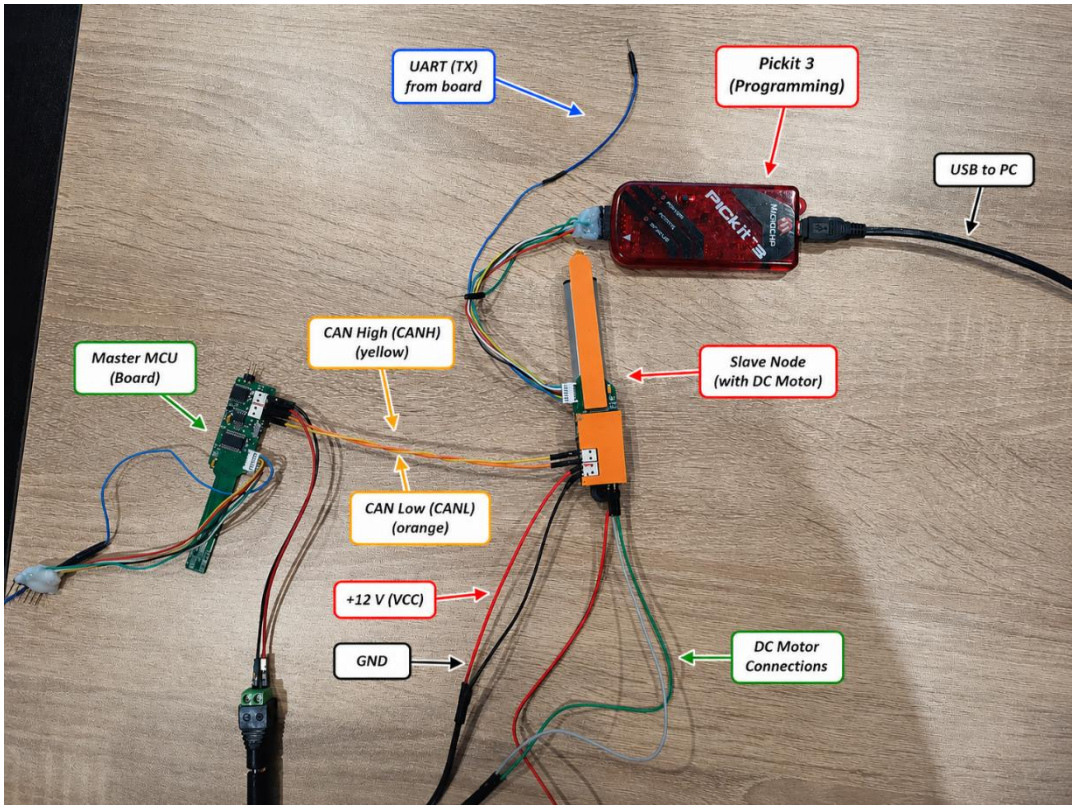

Figure 21 Temporary validation setup

The controller board on the left operated as the master node responsible for command generation and communication supervision, while the board on the right operated as the slave node responsible for local motion control. Communication between both nodes was established through the CAN bus, while UART output was used for diagnostic monitoring during development,

The primary objective of this validation stage was to verify correct operation of the implemented firmware before complete mechanical integration. Particular attention was given to:

- execution of movement commands toward predefined target positions;
- position tracking using encoder feedback;
- homing-state progression and reference handling;
- fault detection and fault-state transitions;
- stopping behavior near targets;
- repeated command execution during continuous operation.

5.1.2. Results and observations

Several observations and implementation-related issues were identified during the validation process.

During repeated positioning tests, oscillatory behavior was observed near the target position under stopping conditions. This behavior was mainly related to the interaction between the stopping tolerance, encoder update timing and movement control logic. As a result additional adjustments were introduced in the firmware to improve stability near the target coordinate and reduce unnecessary corrective movement.

The homing and reference-handling sequence also required further refinement during testing. In particular it was observed that the transition between the release phase and the verification re-approach phase benefited from introducing a short stabilization delay. Without this delay the mechanical system occasionally had insufficient time to settle before the next movement sequence began, which could lead to inconsistent reference detection behavior.

Another important observation concerns encoder scaling and calibration. Since the temporary validation setup differs mechanically from the final stereotactic actuation platform, the encoder scaling coefficients used during preliminary testing were not considered final. The conversion relationship between encoder counts and physical displacement required recalibration after integration with the final mechanical structure.

Special attention was also given to testing abnormal operating conditions. Simulated invalid movement requests, interrupted movement sequences and inconsistent feedback conditions were introduced during testing in order to evaluate the implemented fault-handling mechanisms. In these cases the firmware correctly transferred the system into predefined safe states and prevented continued motion execution under invalid conditions.

One of the most important results of the preliminary validation was confirmation that the developed firmware architecture remained largely independent of the underlying actuator hardware. Although the temporary setup used a DC motor instead of the final stepper-motor-based actuation system, the higher-level axis-control logic and state-machine implementation required only minor modifications. Most hardware-specific functionality remained isolated within the low-level motor and feedback layers.

Although the substitute hardware did not fully reproduce the final stereotactic positioning system, the performed tests confirmed stable operation of the core firmware architecture and provided a practical platform for continued development and debugging.

5.2. CAN Validation

In addition to the validation of the standalone motion-control firmware, the distributed communication architecture of the system was also tested using the available substitute hardware platform. The objective of this stage was to verify communication between the master controller and slave nodes before integration of the final mechanical system.

The validation setup consisted of two PIC18F26K83-based controller boards connected through a CAN interface. The previously used substitute motor and encoder setup was reused during testing in order to evaluate interaction between the communication layer and the implemented axis-control firmware.

Initial testing focused on verification of low-level CAN communication. Simple communication tests using LED indication and periodic message exchange were first performed in order to confirm correct transmission and reception of CAN frames between nodes. These tests were also used to verify basic ECAN peripheral configuration and message handling functionality.

After successful low-level communication verification, the implemented application-level communication protocol was integrated into the system. The master controller transmitted commands to the slave node, while the slave executed the requested operation locally and returned status information through acknowledgment frames.

Particular attention was given to validation of the acknowledgment and retransmission mechanisms implemented within the communication protocol. Each transmitted command contained a sequence number which was returned by the slave node in the corresponding response frame. This allowed the master controller to verify that received responses corresponded to the correct requests.

To evaluate the implemented supervision logic, incorrect sequence values and invalid responses were intentionally introduced during testing. In these situations, the master controller correctly rejected the received messages and activated the retransmission mechanism according to the implemented communication strategy.

Following protocol validation, the communication layer was integrated with the complete axis-control firmware. Movement commands, homing operations and status requests were transmitted through the CAN network and executed by the slave node.

Additional command-validation logic was also verified during testing. The master controller monitored status information returned by the slave node and rejected commands when the axis was busy, not yet homed or reporting a fault condition. This prevented conflicting movement requests and improved overall communication supervision.

Table 7 CAN validation summary

Validation activity	**Result**
CAN frame transmission	**Successful**
CAN frame reception	**Successful**
Command acknowledgement	**Successful**
Sequence-number verification	**Successful**
Retransmission mechanism	**Successful**
Invalid-response rejection	**Successful**
HOME command execution	**Successful**
MOVE_ABSOLUTE command execution	**Successful**
MOVE_RELATIVE command execution	**Successful**
STATUS request handling	**Successful**

The results demonstrate that the developed communication architecture provides reliable command transmission, acknowledgment supervision and status reporting suitable for the distributed motion-control system developed in this work.

5.3. Final Validation

The final validation focused on verifying that the developed distributed motion-control system operated correctly with the implemented master and slave controller hardware. Unlike the preliminary validation stage, which was mainly used for debugging and firmware refinement, the final validation was focused on confirming correct system behavior during repeated operation.

The tests were performed using the CAN-based command structure developed in this work. Movement and configuration commands were transmitted from the master controller to the slave node, while the slave node executed the requested operation locally and returned status information. This allowed the validation to include not only the motor-control and encoder-feedback functions, but also the interaction between the communication layer and the local axis-control firmware.

The main objective of the final validation was not to determine the absolute positioning accuracy of the system. The available measurement equipment, being a regular plastic mechanical ruler, did not allow reliable sub-millimeter accuracy evaluation. For this reason the validation focused on functional correctness, repeatability, encoder consistency and successful execution of motion commands.

Table 8 Summary of final validation activities

Validation activity	**Validation method**	**Result**
Full-step motor operation	Positioning commands in full-step mode	**Successful**
Half-step motor operation	Positioning commands in full-step mode	**Successful**
A-only encoder mode	Position tracking using channel A with channel B for direction	**Successful**
AB quadrature encoder mode	Position tracking using both encoder channels	**Successful**
CAN functionality	As described in section 5.2.	**Successful**
CAN command execution	Motion commands transmitted from master to slave node, incl. relative move, absolute move and homing	**Successful**
Repeatability testing	10 repetitions of different movement cycles, commanded through CAN, measurement with a ruler	**Successful**
EEPROM storage	Parameter storage commanded through CAN, parameters restored after powering off and on, master disconnected	**Successful**

5.3.1. Encoder Calibration and Feedback Verification

Before repeatability testing the encoder scaling factor had to be calibrated. The purpose of this calibration was to establish the relationship between encoder counts and physical displacement of the axis. First the A-only interrupt mode was enabled for simplified testing.

The calibration process started with a chosen encoder scale factor, $0.01\ \text{mm/raw count}$. A movement command for $40\ \text{mm}$ was then transmitted from the master controller through the CAN interface to the slave node. The slave node executed the movement locally and the resulting displacement was measured using the ruler - $10\ \text{mm}$.

The new encoder scale was calculated using the following relationship:

$$new\ encoder\ scale = old\ encoder\ scale \times \frac{measured\ displacement}{commanded\ displacement}$$

$$new\ encoder\ scale = 0.02\ \frac{\text{mm}}{\text{raw count}}$$

This procedure was repeated to verify that the commanded displacement, measured displacement and reported firmware position corresponded as closely as possible within the resolution of the measuring method.

The calibration was not intended to establish high-precision metrological accuracy. Instead, it provided a practical conversion factor between encoder counts and physical movement, which was sufficient for validating the firmware behavior and repeatability of the control system.

5.3.2. Validation of Motor Operating Modes

The motor-control module was validated in both full-step and half-step operation.

Initial validation was performed using full-step excitation. In this mode, the motor successfully executed homing procedures, absolute movement commands and repeated movement sequences. After the full-step implementation was verified, the firmware was reconfigured for half-step operation and the same tests were repeated.

In half-step mode, the motor also completed the commanded movements successfully. The motion was visibly smoother due to the additional intermediate excitation states used in the half-step sequence. The successful operation of both modes confirmed that the motor excitation logic was implemented correctly.

The same higher-level axis-control logic was used for both motor modes. Only the low-level excitation sequence inside the motor module was changed. This confirmed the separation between the motion-control algorithm and the motor actuation implementation.

5.3.3. Validation of Encoder Operating Modes

The encoder-processing module was validated in two operating modes.

The first mode used channel A as the main counting signal. In this configuration, interrupts generated by channel A were used to update the encoder count, while channel B was

sampled to determine the direction of movement. This mode was implemented and tested first because it provided a simpler and more stable basis for early validation.

After successful operation was achieved with the A-only mode, the AB quadrature implementation was tested. In this mode, both encoder channels were used for position tracking. Because quadrature decoding evaluates the transitions of both channels, it provides four times higher counting resolution compared with the single-channel method.

When switching from the A-only implementation to AB quadrature decoding, the encoder scale factor was divided by four. This was necessary because the same physical movement produced four times more encoder counts. After this adjustment, the reported position remained consistent with the physical displacement observed during testing.

$$encoder\ scale\ AB = \frac{encoder\ scale\ A}{4}$$

$$encoder\ scale\ AB = 0.0005\ \frac{\text{mm}}{\text{raw count}}$$

Both encoder modes were tested using homing and movement commands transmitted through the CAN interface. Correct direction detection, position tracking and return to the reference position were observed in both configurations. This confirmed that the higher-level axis-control logic could operate correctly with both encoder-processing methods.

5.3.4. Repeatability Testing

Repeatability testing was the main part of the final validation. The purpose of this test was to verify that repeated execution of the same movement commands resulted in consistent system behavior and no cumulative drift in the encoder feedback.

Before each repeatability test the axis was homed in order to establish a consistent reference position. After homing, predefined movement sequences were transmitted from the master controller to the slave node through the CAN bus.

Several movement patterns were used during testing, including:

- 0 mm → 15 mm → 0 mm → 30 mm → 0 mm;
- 0 mm → 40 mm → 20 mm → 40 mm → 0 mm;
- 0 mm → full range → 0 mm;
- repeated movements between the homed position and a fixed target position.

The repeatability test consisted of ten movement cycles. During each cycle the axis was commanded to move to the selected target positions and return to the reference position.

The test was performed in both full-step and half-step motor modes. In both cases the system completed the repeated movement sequences successfully.

No cumulative drift was observed in the external mechanical position measurement. This indicated stable operation of the encoder-processing logic, homing procedure and axis-control state machine.

As an additional check selected movements to predefined positions, such as 30 mm, were measured using a ruler. The ruler did not provide sufficient resolution for sub-millimeter accuracy evaluation, but repeated movement commands consistently resulted in the same visible physical position within the resolution of the measuring method.

Therefore, the repeatability validation confirmed that the system could reliably execute repeated motion commands and return to the same reference position without observable drift.

In addition to the functional repeatability tests summarized in Table 9, a series of quantitative validation measurements were performed to evaluate homing consistency and positioning repeatability at fixed target positions. A dedicated firmware module (validation_log.c) was developed for quantitative validation. The module recorded encoder counts and position values during repeated homing and positioning cycles. Measurement data were obtained through the corresponding getter functions of the encoder and axis-control modules and stored in an internal log buffer. Following test execution, the recorded values were extracted and processed externally to generate the graphs presented in this section.

** For improved visibility of the observed variations, the vertical axes of the following graphs are presented using a reduced scale and do not start at zero. The selected axis limits were chosen to emphasize the measured repeatability and error characteristics.*

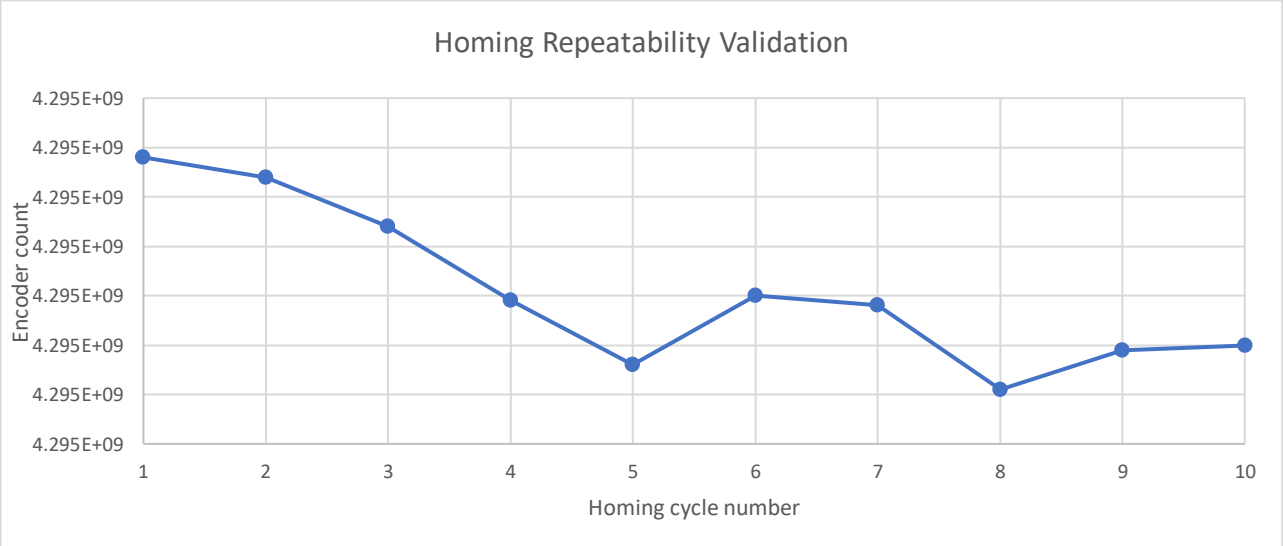

Figure 22 Detected reference position during ten consecutive homing cycles.

To evaluate the consistency of the homing procedure, ten consecutive homing cycles were executed. The detected reference position varied between −4925 and −4878 encoder counts, corresponding to a total spread of approximately 0.094 mm using the calibrated encoder

scale factor. All homing cycles completed successfully and the axis consistently reached the reference position without entering a fault state. The results indicate repeatable operation of the implemented homing algorithm.

Following the homing procedure, the axis was repeatedly commanded to move to a target position of 10 mm. The reported position remained close to the commanded value in all repetitions. The measured positions varied between 9.950 mm and 9.972 mm, corresponding to a maximum observed positioning error of 0.050 mm. No cumulative drift or unstable behavior was observed during testing.

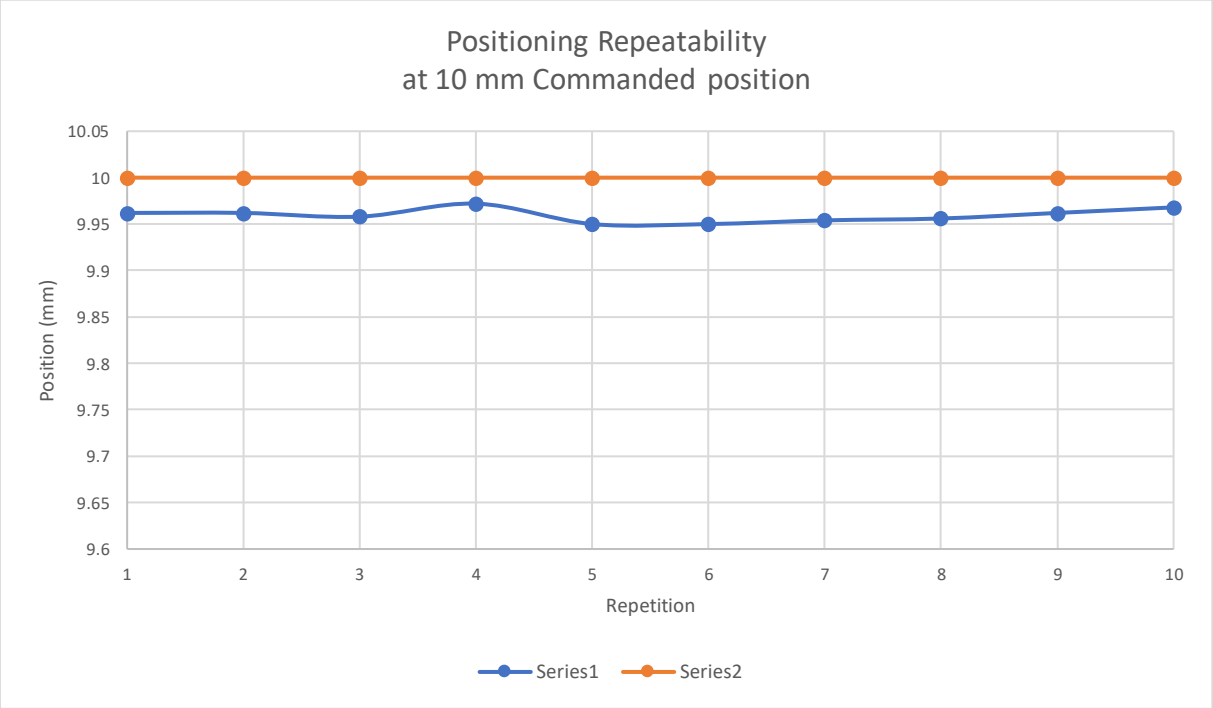

Figure 23 Reported position during ten consecutive commands of 10 mm.

The positioning error for the 10 mm target remained within a narrow band between −0.028 mm and −0.050 mm. The average positioning error was approximately −0.041 mm.

Since all measurements exhibited a similar negative offset, the remaining error appears predominantly systematic rather than random. This behavior suggests that additional calibration could further reduce the observed deviation.

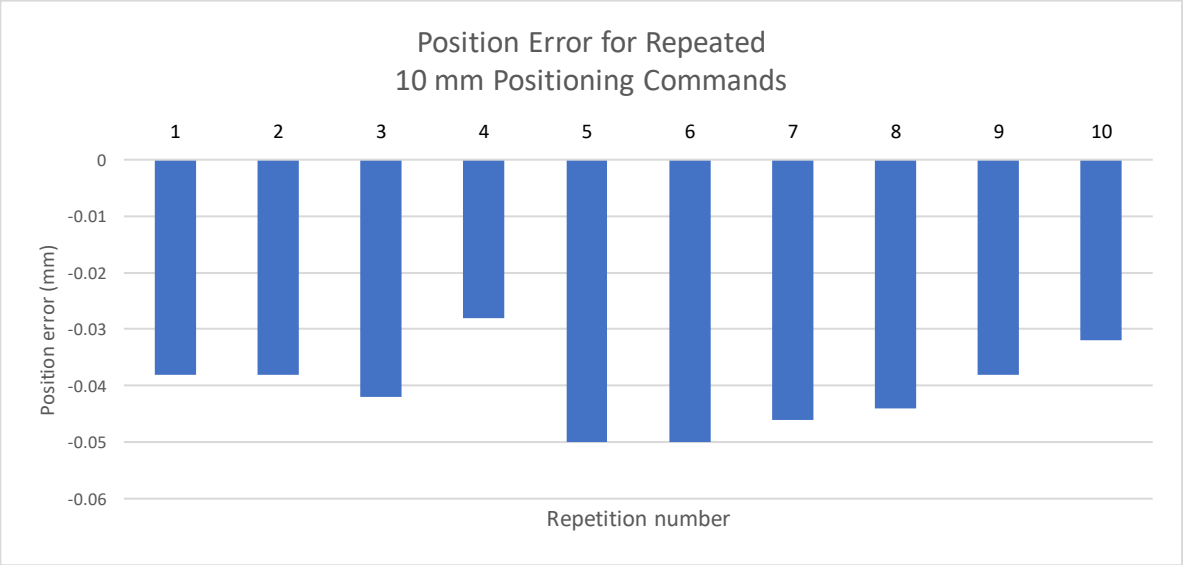

Figure 24 Positioning error observed during ten consecutive commands of 10 mm.

A similar repeatability test was performed using a target position of 20 mm. The reported positions remained consistently close to the commanded value throughout all repetitions. The measured positions varied between 19.950 mm and 19.984 mm. The results confirm stable operation of the motion-control algorithm over a larger displacement range while maintaining repeatable positioning behavior.

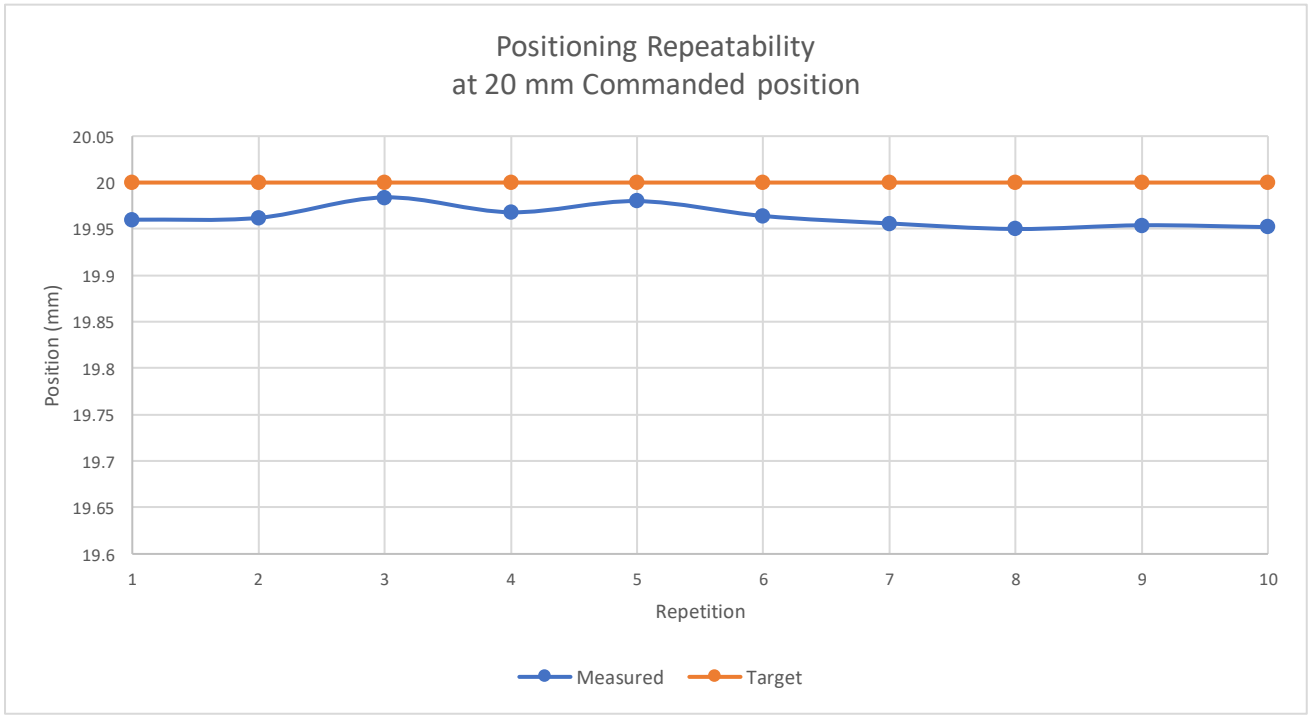

Figure 25 Reported position during ten consecutive commands of 20 mm.

The error measurements obtained at the 20 mm target position showed behavior similar to that observed during the 10 mm test. All measured errors remained below 0.05 mm and no outliers were observed. The consistent error magnitude and absence of significant variation between repetitions further support the repeatability of the developed motion-control system.

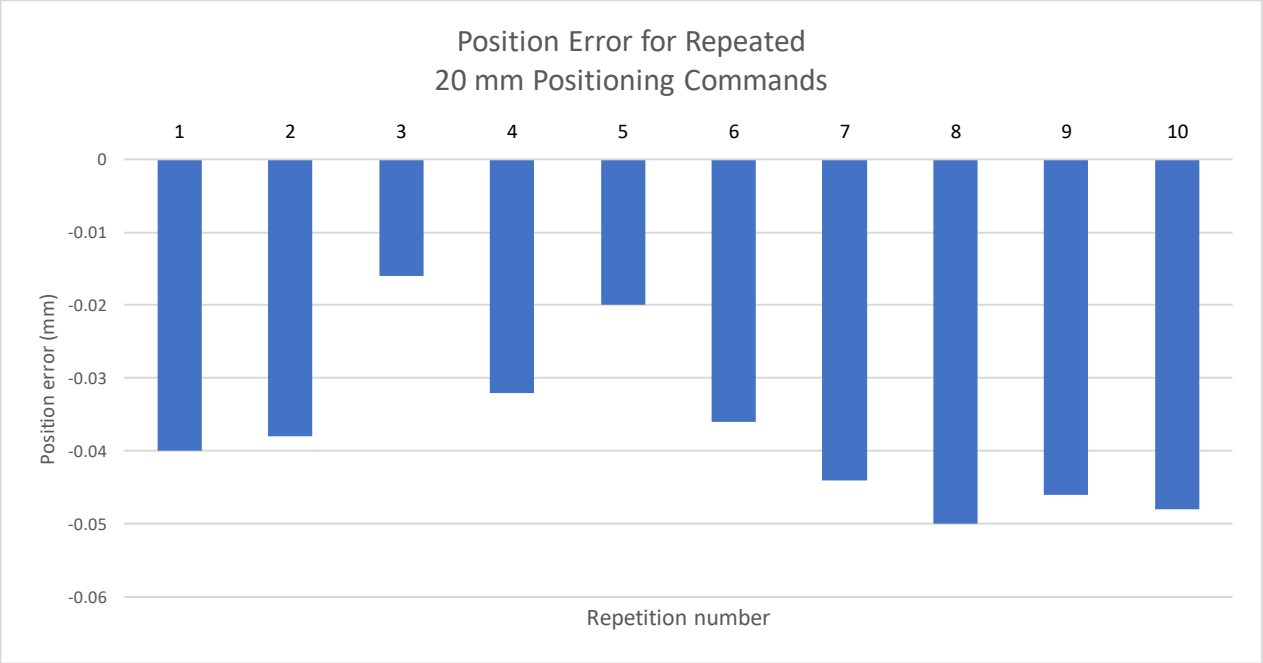

Figure 26 Positioning error observed during ten consecutive commands of 20 mm.

The quantitative measurements presented in Figures 22–26 support the observations obtained during the functional repeatability tests and demonstrate consistent homing behavior, repeatable positioning and stable encoder feedback.

Table 9 Qualitative repeatability validation summary

Verification condition	**Check note**	**Result**
Return to reference position at mechanical 0 - *test performed 10 times* *Both full-step and half-step motor mode; encoder AB quadrature*	Starting at 0 repeated movement commands sent, then last command move to 0 mm.	**Successful - nut returned to 0 20/20 times**

Return to reference position at random mechanical position - *test performed 10 times* *Half-step motor mode; encoder AB quadrature*	Starting at random mechanical position, bypass homing, zero encoder, repeated movement commands sent, then last command move to 0 mm. Initial measured position of the nut must match the final.	**Successful - nut returned to reference position 10/10 times**
Return to reference position at mechanical 0 - **cumulative drift measurement** - *10 cycles - test performed 5 times.* *Both full-step and half-step motor mode; encoder AB quadrature*	Starting at 0 repeated movement commands sent, then last command move to 0 mm. 20 cycles. Check cumulative drift of the reference position after 20 cycles	**Successful - nut returned to reference position without drift 5/5 times**
Return to reference position at preset mechanical position - **cumulative drift measurement** - *10 cycles - test performed 5 times.* *Both full-step and half-step motor mode; encoder AB quadrature*	Starting at a preset mechanical position, bypass homing, zero encoder, repeated movement commands sent, then last command move to 0 mm. 20 cycles. Check cumulative drift of the reference position after 20 cycles	**Successful - nut returned to reference position without drift 5/5 times**
Verification at fixed target - *5 points over full range, 5 times per point*	Send commands for a move to 5 different points over the full range, measure the physical distance	**Successful - no deviation over 1 mm was observed ***

** The displacement measurements used for this validation were verified using a commercially available 300 mm plastic ruler with 1 mm graduations. Since the ruler is not a calibrated measuring instrument, the measurement uncertainty is estimated to be approximately ±0.5 mm to ±1 mm, considering graduation spacing, visual alignment (parallax) and manual reading errors. Consequently the ruler was used only for approximate verification of positioning and repeatability rather than for precise accuracy characterization.*

5.3.5. EEPROM Validation

The EEPROM functionality was validated by storing configuration values in non-volatile memory and verifying that they remained available after restarting the controller.

During testing configuration parameters were modified and saved through the implemented configuration-management mechanism. After storing the values the controller was restarted. During the next initialization the firmware loaded the stored configuration from EEPROM and applied the saved parameters automatically.

This test confirmed that calibration-related parameters could be preserved without recompiling or reprogramming the firmware. This was especially useful during encoder calibration, because adjusted scale values could be stored permanently and reused during later test sessions.

5.3.6. Final Validation Summary

The final validation confirmed the main functions of the developed motion-control firmware. CAN-based command execution, homing, encoder feedback, motor operation, repeatability and EEPROM storage were all verified successfully.

The repeatability measurements presented in Figures 22–26 demonstrated consistent homing behavior and stable positioning performance. Repeated positioning operations at target positions of 10 mm and 20 mm produced errors below 0.05 mm in all measured repetitions, while no cumulative drift was observed during the performed tests.

Although the available measurement equipment did not allow precise characterization of absolute positioning accuracy, the obtained results confirmed reliable execution of motion commands, repeatable reference detection and stable interaction between the implemented software modules.

The validation results show that the developed firmware architecture is suitable for further integration with the complete mechanical system and provides a functional basis for continued improvement of the automated stereotactic positioning system.

6. Conclusion and Future Work

The objective of this bachelor thesis was the development of an actuation and control system for selected axes of a Leksell stereotactic frame, with emphasis on the design and implementation of embedded motion-control firmware. The developed solution combines distributed communication, modular software architecture and encoder-based feedback to provide controlled and repeatable axis positioning.

A distributed master–slave architecture based on CAN communication was successfully implemented. The master node performs command validation, communication supervision and system-level coordination, while individual slave nodes execute all time-critical motion-control tasks locally. This separation improves modularity and simplifies future expansion of the system.

A complete motion-control firmware framework was developed, including an axis state machine, homing procedures, motor control, encoder feedback processing, fault handling and configuration management. The modular software structure separates application logic from hardware-specific implementation details, improving maintainability and allowing adaptation to different hardware configurations with minimal changes to the higher-level control logic.

An encoder-based feedback system was implemented for position estimation, motion verification and homing supervision. The developed homing algorithm enables automatic reference detection and establishment of a repeatable coordinate system. In addition, persistent storage of operational parameters was implemented using the internal EEPROM memory of the microcontroller, allowing configuration values to be retained after power cycling.

Experimental validation confirmed correct operation of the implemented firmware. Motion execution, homing procedures, CAN communication and EEPROM-based configuration storage were successfully verified using dedicated test setups. Quantitative repeatability measurements demonstrated consistent homing behavior and stable positioning performance, with positioning errors remaining below 0.05 mm during repeated positioning operations at fixed target positions. The obtained results demonstrate reliable operation of the developed architecture and validate the proposed approach for embedded distributed motion-control systems.

The developed system provides a functional foundation for future integration into the complete stereotactic positioning platform and demonstrates the applicability of distributed embedded control techniques in precision positioning applications.

Future work

Several opportunities for future development and improvement of the system have been identified during the implementation and validation process.

The current work focuses primarily on the firmware architecture and validation of individual motion-control functions. Future work should include integration and validation of the complete multi-axis system on the final stereotactic frame hardware. Particular attention should be given to coordinated motion of multiple axes and verification of positioning performance under realistic operating conditions. Future validation should include simultaneous operation of multiple coordinated axes in order to evaluate synchronization performance and overall system behavior under realistic stereotactic positioning scenarios.

Additional development may include implementation of advanced synchronization mechanisms between slave nodes, expanded diagnostic capabilities and enhanced fault-detection strategies. The communication protocol could also be extended to support additional configuration parameters and more detailed status reporting.

The current implementation stores key operational parameters in EEPROM memory. Future versions could extend the configuration system to include additional runtime-selectable options, such as motor operating modes and encoder-processing modes, which are currently defined through compile-time configuration settings.

Finally, a more comprehensive evaluation of positioning accuracy, repeatability and long-term reliability could be performed using dedicated metrology equipment. Future validation could employ a CMM or other high-accuracy dimensional measurement systems to determine absolute positioning accuracy, repeatability, backlash effects and overall measurement uncertainty more precisely. Such investigations would provide quantitative performance metrics and support further refinement of the developed motion-control system.

Appendices

A. Function Implementation AXIS_Task(AXIS_t *axis)

```
void AXIS_Task(AXIS_t *axis){
    float absDeltaMm;
    uint16_t moveFrequencyHz;

    if ((axis == 0) || (axis->motor == 0)||(axis->encoder == 0)) return;


    #if (AXIS_ENABLE_SIMULATION == 1u)
    AXIS_RunSimulation(axis);
    #endif
    AXIS_UpdateFeedback(axis);

    if (axis->state == AXIS_STATE_FAULT) {
        AXIS_StopMotor(axis);
        return;
    }

    if (axis->emergencyActive) {
        AXIS_StopMotor(axis);
        axis->state = AXIS_STATE_EMERGENCY;
        return;
    }

    switch (axis->state) {
        case AXIS_STATE_UNINITIALISED: {
            AXIS_StopMotor(axis);
            break;
        }

        case AXIS_STATE_NOT_HOMED: {
            AXIS_StopMotor(axis);
            break;
        }

        case AXIS_STATE_HOMING: {
            AXIS_HandleHoming(axis);
            break;
        }

        case AXIS_STATE_READY: {
            AXIS_StopMotor(axis);
            break;
        }

        case AXIS_STATE_MOVING: {
            axis->positionDeltaMm    =    axis->targetPositionMm    -
axis->currentPositionMm;
            absDeltaMm = AXIS_AbsFloat(axis->positionDeltaMm);
```

```
            if(absDeltaMm <= axis->config.positionToleranceMm){
                AXIS_StopMotor(axis);
                axis->positionDeltaMm = 0.0f;
                axis->state = AXIS_STATE_READY;
                break;
            }

            MOTOR_Enable(axis->motor);

            if(absDeltaMm > axis->config.fastZoneMm) moveFrequencyHz =
axis->config.moveSpeedFastHz;
            else moveFrequencyHz = axis->config.moveSpeedSlowHz;

            if (axis->positionDeltaMm > 0.0f) {
                AXIS_CommandMotor(axis, AXIS_GetDirectionForPositive-
Move(axis), moveFrequencyHz);
            } else {
                AXIS_CommandMotor(axis, AXIS_GetDirectionForNegative-
Move(axis), moveFrequencyHz);
            }

            if(!MOTOR_IsRunning(axis->motor)) {
                MOTOR_Start(axis->motor);
            }

            break;
        }

        case AXIS_STATE_STOPPING: {
            AXIS_StopMotor(axis);

            if(axis->homed) axis->state = AXIS_STATE_READY;
            else axis->state = AXIS_STATE_NOT_HOMED;

            axis->targetPositionMm = axis->currentPositionMm;
            axis->positionDeltaMm = 0.0f;
            axis->homingStep = AXIS_HOMING_IDLE;
            break;
        }

        case AXIS_STATE_EMERGENCY: {
            AXIS_StopMotor(axis);
            break;
        }

        case AXIS_STATE_FAULT: {
            AXIS_StopMotor(axis);
            break;
        }

        default: {
            AXIS_SetFault(axis, AXIS_FAULT_INVALID_STATE);
            break;
        }
    }
```

```
}
```

B. Function Implementation MASTER_AXIS_CLIENT_SendCommand (AXIS_ID_t axisId, SLAVE_NODE_Command_t command, float valueMm)

```
bool MASTER_AXIS_CLIENT_SendCommand(AXIS_ID_t axisId, SLAVE_NODE_Command_t command, float valueMm) {
    CAN_PROTOCOL_CommandFrame_t commandFrame;
    CAN_PROTOCOL_ResponseFrame_t responseFrame;
    CAN_PROTOCOL_Result_t canResult;
    uint8_t retry;
    uint8_t thisSequence;

    if (!MASTER_AXIS_CLIENT_IsAxisValid(axisId)) {
        send_string("REJECT INVALID AXIS\r\n");
        return false;
    }

    if (!MASTER_AXIS_CLIENT_IsCommandValid(command)) {
        send_string("REJECT INVALID CMD\r\n");
        return false;
    }

    if (MASTER_AXIS_CLIENT_CommandMove(command)) {
        if (MASTER_AXIS_CLIENT_IsBusy(axisId)) {
            send_string("REJECT MOVE: AXIS BUSY\r\n");
            return false;
        }

        if(MASTER_AXIS_CLIENT_HasFault(axisId)) {
            send_string("REJECT MOVE: AXIS FAULT\r\n");
            return false;
        }
    }

    thisSequence = sequence++;

    commandFrame.command = command;
    commandFrame.sequence = thisSequence;
    commandFrame.valueMm = valueMm;

    for (retry = 0u; retry < MASTER_AXIS_CLIENT_MAX_RETRIES; retry++)
{
        canResult = CAN_PROTOCOL_SendCommand(masterToAxisId[axisId],
&commandFrame);

        if (canResult != CAN_PROTOCOL_OK) {
            send_string("TX FAIL=");
            send_int16_as_str_to_uart((int16_t)canResult);
            send_string("\r\n");
            continue;
        }
```

```
        send_string("TX AXIS=");
        send_int16_as_str_to_uart((int16_t)axisId);
        send_string(" CMD=");
        send_int16_as_str_to_uart((int16_t)command);
        send_string(" SEQ=");
        send_int16_as_str_to_uart((int16_t)thisSequence);
        send_string(" TRY=");
        send_int16_as_str_to_uart((int16_t)retry);
        send_string("\r\n");

        if (MASTER_AXIS_CLIENT_WaitForAck(axisId, thisSequence, &re-
sponseFrame)) {
            return true;
        }

        send_string("NO ACK RETRY\r\n");
    }
    send_string("COMMAND FAILED: NO ACK\r\n");
    return false;
}
```

References

[1] Elekta AB, "Leksell stereotactic system®: Instructions for use," Stockholm, 2015.

[2] Elekta AB, "Leksell stereotactic system®," 2012. [Online].

[3] Oriental Motor Co. Ltd, Technical manual: Stepper motor edition, 2018.

[4] Monolithic Power Systems, "Stepper Motors Basics: Types, Uses, and Working Principles," [Online]. Available: https://www.monolithicpower.com/en/learning/resources/stepper-motors-basics-types-uses.

[5] Electronics Tutorials, "H-bridge Circuit," [Online]. Available: https://www.electronics-tutorials.ws/io/h-bridge-circuit.html. [Accessed May 2026].

[6] Oriental Motor, "Stepper Motor Basics," [Online]. Available: https://www.orientalmotor.com/stepper-motors/technology/stepper-motor-basics.html. [Accessed May 2026].

[7] P. Acarnley, Stepping motors: a guide to theory and practice, London: The Institution of Engineering and Technology, 2007.

[8] L. E. Frenzel Jr, Handbook of serial communications interfaces, Oxford: Elsevier, 2016.

[9] M. Di Natale, H. Zeng, P. Giusto and A. Ghosal, Understanding and Using the Controller Area Network Communication Protocol: Theory and Practice, New York: Springer, 2012.

[10] Microchip Technology Inc., "PIC18(L)F25/26K83 Data Sheet," 2020.